breedtools: Design, Analysis and Breedbase Storage of Plant Breeding Trials

November 3, 2026

Title Design, Analysis and 'Breedbase' Storage of Plant Breeding Trials

Version 0.1.0

Description Designs and analyses plant breeding field trials, from early generations with a single plot per entry to fully replicated trials, and saves the designs to a 'Breedbase' database. Designs: augmented row-column designs with replicated checks (Federer, Nair and Raghavarao (1975) <doi:10.2307/2529426>), including unequal row and column groups, incomplete blocks, extra check replication and D-optimal check placement (Piepho and Williams (2016) <doi:10.2134/agronj2016.06.0325>); multi-location augmented partially replicated (p-rep) designs built by search or from alpha and alpha(2) arrays (Williams, Piepho and Whitaker (2011) <doi:10.1002/bimj.201000102>); and resolvable spatial row-column designs optimized for row-column and linear variance efficiency, with latinized options (Williams, John and Whitaker (2006) <doi:10.1111/j.1541-0420.2005.00393.x>). Analyses give analysis of variance tables with degrees of freedom estimable from the observed plots and adjusted entry means with standard errors, from fixed models, mixed models with random blocks or random entries (best linear unbiased predictions), genotype-by-location interaction, spatial P-spline models with 'SpATS' (Rodriguez-Alvarez et al. (2018) <doi:10.1016/j.spasta.2017.10.003>), two-stage weighted analysis of multi-location trials with heterogeneous error variances (Mohring and Piepho (2009) <doi:10.2135/cropsci2009.02.0083>), and residual maximum likelihood fits of row-column, linear variance and autoregressive spatial models. Draws field maps, writes field books, checks designs, and saves designs as trials with their plots into a 'Breedbase' breeding program and folder through the Breeding API ('BrAPI'), and later the phenotypic data of those plots by plot and trait name, first-stage plot values as a trait and second-stage genotype values as a 'Breedbase' analysis. Also loads 'Breedbase' phenotype files, checks residual variance homogeneity across environments with Hartley's Fmax test (Hartley (1950) <doi:10.1093/biomet/37.3-4.308>), and groups environments by genotype-by-environment interaction (Lin (1982) <doi:10.1007/BF00276251>).

License GPL-3

URL https://github.com/chris263/breedtools

BugReports https://github.com/chris263/breedtools/issues

Encoding UTF-8

Language en-US

Roxygen list(markdown = TRUE)

RoxygenNote 8.0.0

Depends R (>= 4.1.0)

Imports dplyr,
tidyr,
purrr,
readxl,
leaflet,
htmltools,
tibble,
tools,
graphics,
grDevices,
jsonlite,
stats,
utils

Suggests knitr,
rmarkdown,
lme4,
httr2 (>= 1.1.0),
askpass,
Matrix,
SpATS,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

Contents

breedtools-package 4
add_environment_metadata 6
add_pairwise_environment_metadata 6
analyze_augmented_prep_design 7
analyze_augmented_prep_two_stage 10
analyze_augmented_row_column_design 13
analyze_environment_anova 15
analyze_one_environment 16
analyze_resolvable_row_column 17
augmented_prep_concurrence 19

augmented_prep_design . . . 19
augmented_prep_parameters . . . 23
augmented_row_column_capacity . . . 24
augmented_row_column_design . . . 25
augmented_row_column_layouts . . . 28
breedbase_adjusted_trait . . . 29
breedbase_check . . . 30
breedbase_lists . . . 32
breedbase_login . . . 33
calculate_environment_stratification_anova . . . 34
check_design . . . 34
check_reset . . . 35
check_result . . . 36
check_section . . . 37
check_setup . . . 37
check_summary . . . 38
complete_environment_info . . . 39
detect_design . . . 39
environment_summary_by_group . . . 40
first_stage_values . . . 41
get_compatible_groups . . . 42
get_maximal_groups . . . 42
guess_column . . . 43
hartley_critical_sim . . . 44
hartley_fmax_test . . . 44
lin_error_mse . . . 45
lin_group_environments . . . 46
lin_ss_ge . . . 47
lin_test_group . . . 47
make_environment_groups . . . 48
map_environment_groups . . . 49
phenotype_template . . . 50
plot_augmented_prep_design . . . 51
plot_augmented_row_column_design . . . 53
plot_resolvable_row_column_design . . . 55
prediction_accuracy . . . 56
prepare_breedbase_pheno . . . 57
prepare_design_factors . . . 58
prepare_environment_stratification_data . . . 58
print.augmented_prep_analysis . . . 59
print.augmented_prep_two_stage . . . 60
print.augmented_row_column_analysis . . . 60
print.resolvable_row_column_analysis . . . 61
read_breedbase_file . . . 61
read_locations . . . 62
resolvable_row_column_design . . . 64
resolvable_row_column_efficiency . . . 66
run_breedbase_environment_stratification . . . 67

run_breedbase_variance_check 68
run_environment_stratification 70
save_breedbase 71
save_breedbase_analysis 74
save_breedbase_first_stage 76
save_breedbase_phenotypes 77
second_stage_values 79

Index **81**

`breedtools-package` *breedtools: design, analysis and 'Breedbase' storage of plant breeding trials*

Description

breedtools designs and analyses plant breeding field trials, from early generations with a single plot per entry to fully replicated trials, and saves the designs into a 'Breedbase' database. The tutorial, `vignette("breedtools")`, walks through every design and analysis.

Which design?

- One plot per entry, one location: augmented row-column design with replicated checks.
- One plot per entry per location, several locations: augmented partially replicated (p-rep) design, without checks.
- Every entry in every replicate: resolvable (spatial) row-column design.

Augmented row-column designs

Unreplicated entries plus replicated checks in rows, columns and blocks (Federer, Nair and Raghavarao, 1975; Piepho and Williams, 2016), with equal or unequal row and column groups, complete or incomplete blocks of checks, and Latin or D-optimal check placement that keeps checks apart.

- Plan: `augmented_row_column_layouts()`, `augmented_row_column_capacity()`
- Design: `augmented_row_column_design()`
- Map: `plot_augmented_row_column_design()`
- Analysis (fixed, random blocks, random entries, SpATS spatial): `analyze_augmented_row_column_design()`

Augmented p-rep designs

Multi-location trials in which a share of the entries is repeated at each location (Williams, Piepho and Whitaker, 2011), built by search or from alpha and alpha(2) arrays.

- Plan: `augmented_prep_parameters()`
- Design: `augmented_prep_design()`, `augmented_prep_concurrence()`
- Map: `plot_augmented_prep_design()`
- Analysis (with replicates and fixed or random genotype x location): `analyze_augmented_prep_design()`
- Two-stage weighted analysis with block or SpATS spatial stage 1: `analyze_augmented_prep_two_stage()`

Resolvable row-column designs

Fully replicated designs optimized for row-column and spatial (linear variance) efficiency, with latinized options (Williams, John and Whitaker, 2006).

- Design: `resolvable_row_column_design()`
- Efficiency, neighbours, AR criterion: `resolvable_row_column_efficiency()`
- Map: `plot_resolvable_row_column_design()`
- Analysis (RC, LV + RC and AR + RC by REML): `analyze_resolvable_row_column()`

Saving to Breedbase

Designs are saved as trials with their plots into a breeding program and folder through BrAPI, after checking every name; after harvest, the phenotypic data are saved to the same plots.

- Log in: `breedbase_login()`
- Look up names: `breedbase_programs()`, `breedbase_folders()`, `breedbase_locations()`
- Check location and accessions before designing: `breedbase_check()`
- Save (with a dry run first): `save_breedbase()`
- Phenotypes: `phenotype_template()` (sheet with the plot names), `save_breedbase_phenotypes()` (values by plot and trait name)
- Analysis results: `first_stage_values()` and `save_breedbase_first_stage()` (plot values as a trait of the field plots, trait made by `breedbase_adjusted_trait()`); `second_stage_values()` and `save_breedbase_analysis()` (genotype values as a Breedbase analysis)

Multi-environment trials from Breedbase

- Read files: `read_breedbase_file()`, `prepare_breedbase_pheno()`
- Error variances (Hartley's Fmax): `run_breedbase_variance_check()`, `hartley_fmax_test()`
- Environment groups (Lin, 1982): `run_environment_stratification()`, `lin_group_environments()`
- Maps: `read_locations()`, `map_environment_groups()`

Checks

`check_setup()`, `check_design()`, `check_result()`, `check_summary()`, `prediction_accuracy()`.

Author(s)

Maintainer: Christiano Simoes <ccsimoes263@gmail.com>

Authors:

- Christiano Simoes <ccsimoes263@gmail.com>

See Also

Useful links:

- https://github.com/chris263/breedtools
- Report bugs at https://github.com/chris263/breedtools/issues

add_environment_metadata

Add environment metadata to result table

Description

Add environment metadata to result table

Usage

```
add_environment_metadata(results, env_info, env_col = "environment")
```

Arguments

results	Result data frame.
env_info	Environment metadata table.
env_col	Name of the environment column in results.

Value

Joined data frame.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

membership <- data.frame(
  group_id = "Group_1",
  environment = prepared$env_info$environment[1:2]
)
add_environment_metadata(membership, prepared$env_info)
```

add_pairwise_environment_metadata

Add environment metadata to pairwise Lin results

Description

Add environment metadata to pairwise Lin results

Usage

```
add_pairwise_environment_metadata(pairwise, env_info)
```

Arguments

pairwise	Pairwise result table.
env_info	Environment metadata table.

Value

Pairwise table with metadata.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

lin <- lin_group_environments(prepared$data)
add_pairwise_environment_metadata(lin$pairwise, prepared$env_info)
```

---

analyze_augmented_prep_design

Analyze an augmented p-rep design

Description

Fits the fixed-effects incomplete-block model: `y = mean + location + block(location) + genotype + error`. For resolvable designs (a `replicate` column with more than one replicate per location) the model is `y = mean + location + replicate(location) + block(replicate, location) + genotype + error`. With `genotype_by_location = "fixed"` (or `TRUE`) a fixed genotype-by-location interaction is added after genotypes; the error is then estimated only from the p-rep entries, as in a per-location analysis with pooled error (Williams, Piepho and Whitaker, 2011, Section 3). With `genotype_by_location = "random"` the interaction is a random effect, as recommended by the paper when locations are a sample of environments, and the model is fitted by REML with 'lme4'. Sums of squares are sequential, in the order above. Degrees of freedom in the formatted ANOVA are those estimable from the observed (non-missing) plots, so F-tests stay valid when plots are missing. For a complete, connected design they equal the design formulas reported in `anova_df`, which use total plots (`n`), locations (`l`), incomplete blocks (`b`), and genotypes (`v`). Terms with a single level in the data (for example the location term in a single-location trial) are left out of the model and shown with 0 df.

Usage

```
analyze_augmented_prep_design(
  data,
  response,
  genotype_col = "all_entries",
  location_col = "location",
```

```
  block_col = "block",
  entry_type_col = "entry_type",
  extra_repeat_col = "is_extra_repeat",
  repeated_at_col = "repeated_at",
  control_col = "is_control",
  replicate_col = "replicate",
  genotype_by_location = FALSE
)
```

Arguments

data	A complete augmented p-rep design data frame, usually design$design from augmented_prep_design(), or an object returned by augmented_prep_design(), with a response column added.
response	Name of the numeric response column.
genotype_col	Name of the genotype/treatment column.
location_col	Name of the location column.
block_col	Name of the incomplete block column. Block numbers may repeat across locations; blocks are treated as nested within locations.
entry_type_col	Optional entry type column. Used only for genotype summaries.
extra_repeat_col	Optional extra-repeat indicator column. Used only for genotype summaries.
repeated_at_col	Optional column identifying where each entry is repeated. Used only for genotype summaries.
control_col	Optional control flag column. Used only for genotype summaries.
replicate_col	Optional replicate (superblock) column. When present and a location has more than one replicate, replicates nested within locations are fitted, and blocks are nested within replicates. Set to NULL to ignore it.
genotype_by_location	Genotype-by-location interaction in a multi-location analysis: "none" (default, or FALSE), "fixed" (or TRUE), or "random" (needs the 'lme4' package). With "fixed", adjusted means are averaged over locations. With "random", adjusted means are the genotype BLUEs from the mixed model, variance_components gives the REML estimates, and the ANOVA table is the fixed-interaction table, whose genotype-by-location and error mean squares show the size of the interaction.

Value

An object of class augmented_prep_analysis: a list with

input, formula, model The column mapping, model formula, and the lm fit.

anova, anova_df, anova_formatted Sequential ANOVA (estimable df), design df formulas, and a formatted ANOVA table.

plot_results The input data with fitted values and residuals.

genotype_summary One row per genotype with raw, fitted, and adjusted means and the standard error of the adjusted mean.

adjusted_means Genotypes ranked by adjusted mean (best first).

adjusted_means_vcov Covariance matrix of the estimable adjusted means, used by `analyze_augmented_prep_two_stage` for weighting.

diagnostics Error df, residual standard deviation, R-squared, `mean_sed` (mean standard error of a difference between two adjusted means), `lsd_5pct` (the matching least significant difference), and flags for rank deficiency or a saturated fit.

variance_components, mixed_model For `genotype_by_location = "random"`: REML variance components and the `lmerMod` fit; otherwise `NULL`.

message A note when F-tests or adjusted means are not estimable.

Adjusted means

Genotype effects are fixed, so the adjusted means are best linear unbiased estimates (BLUEs): the genotype effect plus the average location and block effect over all observed plots, i.e. the expected value of the genotype in an average plot of the trial. They remove the advantage or penalty of the blocks and locations where each entry happened to be planted, so use `adjusted_mean` (not `raw_mean`) to rank and select entries. Differences between adjusted means are the same as those from `emmeans` on the fitted model. With `genotype_by_location = "fixed"` the genotype-by-location effects are averaged over locations, weighting each location by its share of the observed plots. With `"random"`, standard errors come from the REML covariance of the fixed effects, which includes the genotype-by-location variance; `lsd_5pct` uses the error df of the fixed-interaction model as an approximation.

Per-location analysis

Williams, Piepho and Whitaker (2011) suggest analysing each location with an incomplete-block model. Pass the rows of one location, e.g. `analyze_augmented_prep_design(data[data$location == "A", ], "yield")`. `augmented_prep_design()` keeps the blocks of every location connected, so this is estimable whenever the location has enough p-rep plots.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(8)),
  locations = c("Loc1", "Loc2"),
  block_size = 4,
  n_duplicate_per_location = c(Loc1 = 2, Loc2 = 2),
  n_candidates = 3,
  seed = 123
)

design_data <- design$design
design_data$yield <- rnorm(nrow(design_data), mean = 100, sd = 10)
analysis <- analyze_augmented_prep_design(design_data, response = "yield")
analysis
head(analysis$adjusted_means)
```

`analyze_augmented_prep_two_stage` *Two-stage analysis of a multi-location augmented p-rep trial*

Description

Analyses each location separately (stage 1) and combines the per-location adjusted means across locations (stage 2), weighting them by their precision. This is the approach suggested by Williams, Piepho and Whitaker (2011, Section 3) for adding spatial error components and for heterogeneous error variances across locations: every location gets its own error variance (and, optionally, its own spatial trend), and the stage-2 weights carry that information into the combined means (Möhring and Piepho, 2009).

Usage

```
analyze_augmented_prep_two_stage(
  data,
  response,
  stage1 = c("blocks", "spatial"),
  weighting = c("full", "diagonal", "none"),
  genotype_by_location = c("random", "none"),
  genotype_col = "all_entries",
  location_col = "location",
  block_col = "block",
  replicate_col = "replicate",
  row_col = "row",
  col_col = "col",
  spatial_nseg = NULL,
  spatial_random_blocks = TRUE
)
```

Arguments

`data`	A design data frame (usually `design$design` from `augmented_prep_design()`), or a design object, with a response column.
`response`	Name of the numeric response column.
`stage1`	Stage-1 model: `"blocks"` or `"spatial"`.
`weighting`	Stage-2 weighting: `"full"`, `"diagonal"`, or `"none"`.
`genotype_by_location`	`"random"` (default) to estimate a genotype-by-location variance in stage 2, or `"none"` to omit it.
`genotype_col`, `location_col`, `block_col`, `replicate_col`	Column names, as in `analyze_augmented_prep_design()`.
`row_col`, `col_col`	Field row and column coordinate columns, used by `stage1 = "spatial"`.

`spatial_nseg` Optional number of P-spline segments for columns and rows, `c(col, row)`. Defaults to half the number of distinct columns and rows of each location (at least 3, at most 40), which is much faster than one segment per row or column and usually as accurate.

`spatial_random_blocks`
Logical. If `TRUE` (default), incomplete blocks are included as a random effect in the spatial model.

Value

An object of class `augmented_prep_two_stage`, a list with

adjusted_means Combined genotype means ranked best first, with standard errors and the number of locations.

variance_components Stage-2 REML genotype-by-location variance.

locations One row per location: stage-1 model, plots, genotypes, residual variance and df, mean SED and, for spatial fits, the effective dimension of the spatial surface.

heterogeneity Hartley's Fmax ratio of the stage-1 residual variances with a simulated 5% critical value.

stage1_means The stage-1 genotype means and standard errors per location.

stage1_fits The stage-1 fits (analysis objects or `SpATS` objects), named by location.

diagnostics Settings, number of stage-2 means, REML log-likelihood, mean SED and approximate 5% LSD.

Stage 1

`stage1 = "blocks"` The incomplete-block model of `analyze_augmented_prep_design()` at each location (replicates and blocks fixed, genotypes fixed).

`stage1 = "spatial"` A spatial model fitted with the 'SpATS' package (Rodríguez-Álvarez et al., 2018): genotypes fixed, a smooth two-dimensional P-spline surface over the field rows and columns (`PSANOVA`), replicates fixed and incomplete blocks random. It needs the `row` and `col` field coordinates (use `field_cols` in `augmented_prep_design()`).

The stage-1 genotype means are BLUEs with their full covariance matrix. Each location needs some error degrees of freedom, i.e. enough p-rep entries for its blocks. In spatial fits part of the plot-to-plot noise is taken up by the smooth surface, so the reported residual variance is somewhat smaller than in the block model; the stage-1 covariance matrix accounts for this, and the Hartley test on spatial residual variances is approximate.

Stage 2

The stage-1 means $\bar{y}_{gl}$ are modelled as $\bar{y}_{gl} = \mu + \lambda_l + \gamma_g + (\gamma\lambda)_{gl} + e_{gl}$, with fixed locations and genotypes, random genotype-by-location effects with variance σ^2_{GL} (estimated by REML) and stage-1 errors e whose covariance is taken from stage 1:

`weighting = "full"` the full stage-1 covariance matrix of each location (most accurate);

`weighting = "diagonal"` only the stage-1 variances (weights $1/se^2$);

`weighting = "none"` an unweighted analysis of the stage-1 means with a common error, which ignores heterogeneity (for comparison).

Genotype adjusted means average the location effects with equal weights.

References

Williams, E., Piepho, H.-P. and Whitaker, D. (2011). Augmented p-rep designs. Biometrical Journal 53, 19-27. doi:10.1002/bimj.201000102

Möhring, J. and Piepho, H.-P. (2009). Comparison of weighting in two-stage analysis of plant breeding trials. Crop Science 49, 1977-1988. doi:10.2135/cropsci2009.02.0083

Rodríguez-Álvarez, M. X., Boer, M. P., van Eeuwijk, F. A. and Eilers, P. H. C. (2018). Correcting for spatial heterogeneity in plant breeding experiments with P-splines. Spatial Statistics 23, 52-71. doi:10.1016/j.spasta.2017.10.003

See Also

`analyze_augmented_prep_design()` for the one-stage analysis.

Examples

```
design <- augmented_prep_design(
  treatments = sprintf("G%02d", 1:60),
  locations = c("A", "B", "C"),
  method = "alpha",
  block_size = 6,
  field_cols = 9,
  seed = 1
)
trial <- design$design
set.seed(2)
# Location C is twice as noisy as A and B.
trial$yield <- 50 + stats::rnorm(nrow(trial), sd = ifelse(trial$location == "C", 4, 2))

fit <- analyze_augmented_prep_two_stage(trial, response = "yield")
fit
fit$locations


if (requireNamespace("SpATS", quietly = TRUE)) {
  spatial <- analyze_augmented_prep_two_stage(trial, "yield", stage1 = "spatial")
  spatial$locations
}
```

analyze_augmented_row_column_design
Analyze an augmented row-column design

Description

Fits the fixed-effects model: `y = mean + rowgroup + colgroup + rowgroup:colgroup + rowgroup:row + colgroup:col + genotype + error`. Sums of squares are sequential, in the order above. Degrees of freedom are the ranks actually estimable from the observed (non-missing) plots. For a complete design they equal the augmented row-column formulas, reported in `anova_df`, which use field rows (`k`), field columns (`s`), row groups (`gk`), column groups (`gs`), check cultivars (`vc`), and unreplicated entries (`ve`): row groups `gk - 1`, rows within row groups `k - gk`, column groups `gs - 1`, columns within column groups `s - gs`, blocks `(gk - 1)(gs - 1)`, genotypes `vc + ve - 1`, and error as the remainder of `k * s - 1`.

Usage

```
analyze_augmented_row_column_design(
  data,
  response,
  genotype_col = "all_entries",
  rowgroup_col = "rowgroup",
  colgroup_col = "colgroup",
  row_col = "row",
  col_col = "col",
  type_col = "type",
  control_col = "is_control",
  random_blocks = FALSE,
  entries = c("fixed", "random"),
  spatial = FALSE,
  spatial_nseg = NULL
)
```

Arguments

`data`	A complete augmented row-column design data frame, usually `design$design` from `augmented_row_column_design()`, with a response column added.
`response`	Name of the numeric response column.
`genotype_col`	Name of the genotype/treatment column.
`rowgroup_col`	Name of the row-group column.
`colgroup_col`	Name of the column-group column.
`row_col`	Name of the field row column.
`col_col`	Name of the field column column.
`type_col`	Optional entry type column. Used only for genotype summaries.

`control_col` Optional control flag column. Identifies the checks for `entries = "random"`; otherwise used only for genotype summaries.

`random_blocks` Logical. If `TRUE`, rows within row groups, columns within column groups and blocks are random effects, which recovers inter-block information (Wolfinger et al., 1997; Möhring et al., 2015). Row and column groups stay fixed. Fitted by REML with 'lme4'.

`entries` `"fixed"` (default) or `"random"`. With `"random"`, checks are fixed and the unreplicated entries are random effects from a common breeding population (Piepho, Williams and Fleck, 2006); entry values are then BLUPs. Fitted by REML with 'lme4'.

`spatial` Logical. If `TRUE`, the plot errors get a spatial add-on: a two-dimensional P-spline surface over rows and columns fitted with the 'SpATS' package, with row and column groups fixed and rows, columns and blocks random. Genotypes are fixed.

`spatial_nseg` Optional number of P-spline segments for columns and rows, `c(col, row)`. Defaults to half the number of columns and rows (at least 3, at most 40).

Details

Unreplicated entries each fit their own plot exactly, so the model is fitted without a dense genotype design matrix: the residual comes from the replicated (check) plots alone. `model` is the corresponding `lm` fit on the check plots, with aliased columns removed. This keeps the analysis fast for large trials.

Value

An object of class `augmented_row_column_analysis`: a list with

input, formula, model The column mapping, model formula, and an `lm` fit on the check plots.

anova, anova_df, anova_formatted Sequential ANOVA (estimable df), design df formulas, and a formatted ANOVA table.

plot_results The input data with fitted values and residuals.

genotype_summary One row per genotype with raw, fitted, and adjusted means and the standard error of the adjusted mean.

adjusted_means Genotypes ranked by adjusted mean (best first).

diagnostics Error df, residual standard deviation, R-squared, `mean_sed` (mean standard error of a difference between two adjusted means), `lsd_5pct` (the matching least significant difference), and flags for rank deficiency or a saturated fit.

variance_components, mixed_model For mixed or spatial fits: the REML variance components and the `lmerMod` or `SpATS` fit; otherwise `NULL`.

message A note when F-tests or adjusted means are not estimable.

Mixed and spatial models

The ANOVA table always comes from the fixed-effects model of Piepho and Williams (2016, Eq. 1). With `random_blocks`, `entries = "random"` or `spatial`, the adjusted means, their standard

errors, `mean_sed` and `lsd_5pct` come from the corresponding REML fit instead. Standard errors are exact prediction standard errors from the mixed-model equations for the estimated variance components; `lsd_5pct` uses the fixed-model error df as an approximation.

Adjusted means

With the default fixed-effects model, genotype effects are fixed, so the adjusted means are best linear unbiased estimates (BLUEs). Each is the genotype effect plus the average row, column, and block effect over all observed plots, i.e. the expected value of the genotype in an average plot of the field. For an unreplicated entry this is its observed value corrected for the row, column, and block effects of its plot, estimated from the checks; its standard error combines the plot error and the uncertainty of that correction. Use `adjusted_mean` (not `raw_mean`) to rank and select entries.

Examples

```
design <- augmented_row_column_design(
  treatments = paste0("G", seq_len(108)),
  controls = paste0("Check", seq_len(4)),
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 3,
  seed = 123
)

design_data <- design$design
design_data$yield <- rnorm(nrow(design_data), mean = 100, sd = 10)
analysis <- analyze_augmented_row_column_design(design_data, response = "yield")
analysis
head(analysis$adjusted_means)
```

`analyze_environment_anova` *Run individual ANOVA by environment*

Description

Run individual ANOVA by environment

Usage

```
analyze_environment_anova(pheno)
```

Arguments

pheno	Standardized phenotype data from `prepare_breedbase_pheno()`.

Value

A tibble with one ANOVA summary row per environment.

Examples

```
df <- data.frame(
  environment = factor(rep(c("Loc1", "Loc2"), each = 8)),
  genotype = factor(rep(rep(c("G1", "G2"), each = 4), times = 2)),
  replicate = factor(rep(rep(c(1, 2), each = 2), times = 4)),
  block = factor(1),
  value = c(10, 11, 12, 13, 14, 15, 16, 17,
            15, 16, 17, 18, 19, 20, 21, 22)
)

analyze_environment_anova(df)
```

---

analyze_one_environment

Analyze one environment using ANOVA

Description

Runs an individual ANOVA for one environment. If more than one replicate is present, the model is `value ~ replicate + genotype`; otherwise, the model is `value ~ genotype`.

Usage

```
analyze_one_environment(df)
```

Arguments

`df`	A data frame containing one environment with columns `environment`, `genotype`, `replicate`, and `value`.

Value

A tibble with ANOVA summary statistics.

Examples

```
df <- data.frame(
  environment = "Loc1",
  genotype = factor(rep(c("G1", "G2", "G3"), times = 2)),
  replicate = factor(rep(1:2, each = 3)),
  value = c(10, 12, 15, 11, 13, 14)
)

analyze_one_environment(df)
```

`analyze_resolvable_row_column` *Analyze a resolvable row-column trial with RC, LV + RC or AR + RC models*

Description

Fits the variance models of Williams, John and Whitaker (2006) by residual maximum likelihood (REML), with replicates and treatments fixed:

$$E(y) = R\pi + X\tau, \quad var(y) = \sigma^2 V.$$

`model = "rc"` The standard resolvable row-column mixed model: $V = I + \eta_R P_R + \eta_C P_C$, i.e. random rows and columns within replicates.

`model = "lv"` The two-dimensional linear variance model (LV + RC, Eq. 2): $V = I + \eta_R P_R + \eta_C P_C - (\phi_R F_R + \phi_C F_C + \phi_{RC} F_{RC})$, with LV matrices for plots in the same row, in the same column, and in neither. `lv_terms` chooses which ϕ terms are fitted and `equal_phi = TRUE` uses one ϕ for rows and columns (square plots).

`model = "ar"` Rows and columns plus a separable first-order autoregressive process (AR + RC, Eq. 4): $V = I + \eta_R P_R + \eta_C P_C + \psi D(\rho_C) \otimes D(\rho_R)$ within replicates.

In every model the identity term is the independent error ("nugget"). With `compare = TRUE` the nested sequence RC, LV with one ϕ, LV with ϕ_R, ϕ_C, LV with ϕ_{RC} as well, and AR + RC are all fitted and compared by REML deviance (likelihood-ratio tests for the nested LV terms, and AIC), which is the sequential fitting process the paper recommends for deciding whether spatial terms are needed.

Usage

```
analyze_resolvable_row_column(
  data,
  response,
  model = c("rc", "lv", "ar"),
  lv_terms = c("phi_r", "phi_c"),
  equal_phi = FALSE,
  long_col = NULL,
  compare = TRUE,
  genotype_col = "all_entries",
  replicate_col = "replicate",
  row_col = "row_in_rep",
  col_col = "col_in_rep"
)
```

Arguments

`data`	A design data frame (for example `design$design` from `resolvable_row_column_design()`) or a design object, with a response.

`response`	Name of the numeric response column.
`model`	Variance model reported in `adjusted_means`: `"rc"`, `"lv"`, or `"ar"`.
`lv_terms`	For `model = "lv"`, the spatial terms to fit: any of `"phi_r"`, `"phi_c"`, `"phi_rc"`.
`equal_phi`	Logical. For `model = "lv"`, use a single ϕ for rows and columns.
`long_col`	Optional name of a column with long-column (or long-row) labels to fit as fixed effects, as in latinized designs.
`compare`	Logical. If `TRUE`, also fit and compare the sequence of models.
`genotype_col`, `replicate_col`, `row_col`, `col_col`	Column names.

Details

For latinized designs the long columns (or long rows) can be fitted as fixed effects with `long_col`.

Value

An object of class `resolvable_row_column_analysis`, a list with

plot_results One row per plot: the data, the fitted value (fixed effects plus the predicted row, column and spatial effects) and the residual $V^{-1}(y - X\hat{\beta})$.

adjusted_means Treatment BLUEs (average over replicates) with standard errors, ranked best first.

variance_parameters The REML estimates: σ^2, the η, ϕ, ψ and ρ parameters, and the implied row and column variance components $\sigma^2\eta_R/s$ and $\sigma^2\eta_C/k$.

model_comparison With `compare = TRUE`, the deviance table.

anova Intra-block ANOVA from the fixed model `replicate + rows(replicate) + columns(replicate) + treatment`.

diagnostics REML deviance, number of variance parameters, mean SED and approximate 5% LSD.

References

Williams, E. R., John, J. A. and Whitaker, D. (2006). Construction of resolvable spatial row-column designs. Biometrics 62, 103-108. doi:10.1111/j.15410420.2005.00393.x

Examples

```
design <- resolvable_row_column_design(paste0("T", 1:20), 3, 4, 5,
                                       n_starts = 1, n_iter = 300, seed = 1)
trial <- design$design
set.seed(2)
trial$y <- 10 + 0.3 * trial$row + stats::rnorm(nrow(trial))
fit <- analyze_resolvable_row_column(trial, "y", model = "lv")
fit$model_comparison
head(fit$adjusted_means)
```

augmented_prep_concurrence *Calculate pairwise concurrence for a p-rep design*

Description

Counts how many times each pair of entries appears together in the same incomplete block across all locations.

Usage

```
augmented_prep_concurrence(design, treatments = NULL)
```

Arguments

design	A p-rep design data frame from augmented_prep_design().
treatments	Optional treatment names defining matrix order.

Value

A symmetric integer matrix of pairwise block concurrences.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2"),
  block_size = 4,
  n_candidates = 5,
  seed = 1
)
conc <- augmented_prep_concurrence(design$design)
table(conc[upper.tri(conc)])
```

augmented_prep_design *Generate an augmented p-rep design*

Description

Creates a multi-location augmented partially replicated (p-rep) design (Williams, Piepho and Whitaker, 2011). Instead of replicated checks, a subset of the test entries is replicated at each location: every entry is planted once at every location, and the entries chosen for p-rep at a location get extra plots there. All plots of a location are arranged in incomplete blocks, chosen so that pairs of entries share a block at most once across the whole trial ((0,1)-designs).

Usage

```
augmented_prep_design(
  treatments,
  locations,
  block_size = 8,
  prep_prop = NULL,
  n_duplicate_per_location = NULL,
  replications_per_repeated_entry = 2,
  n_blocks_per_location = NULL,
  resolvable = FALSE,
  n_candidates = 1000,
  seed = NULL,
  field_cols = NULL,
  rows_per_block = NULL,
  blocks_per_row = NULL,
  serpentine = TRUE,
  output_file = NULL,
  method = c("search", "alpha", "alpha2"),
  n_prep_per_block = NULL,
  n_efficiency_candidates = 5
)
```

Arguments

`treatments`	Character vector of entry names.
`locations`	Character vector of location names.
`block_size`	Maximum final incomplete block size. Can be a single integer or a named vector indexed by location (the algebraic methods use its smallest value). Actual block sizes may differ by one plot when totals do not divide evenly.
`prep_prop`	Optional proportion of entries to duplicate per location (`method = "search"` only). If `NULL`, entries are split across locations so each entry is duplicated at approximately one location.
`n_duplicate_per_location`	Optional number of entries duplicated at each location (`method = "search"` only). Can be a single integer, a vector matching `locations`, or a named vector indexed by location.
`replications_per_repeated_entry`	Total number of plots for entries selected for p-rep replication at a location. The default `2` gives one base plot plus one extra plot. Can be a single integer or a named vector indexed by location. Must be 2 for the algebraic methods.
`n_blocks_per_location`	Optional number of incomplete blocks at each location (`method = "search"` only). If `NULL`, the number of blocks is the minimum needed so no block is larger than `block_size`. Can be a single integer or a named vector indexed by location.
`resolvable`	Logical (`method = "search"`). If `TRUE`, each location is laid out as a resolvable design: the blocks are grouped into `replications_per_repeated_entry`

replicates, each replicate holds every p-rep entry of that location exactly once plus an even share of the unreplicated entries, and all replicates have the same number of blocks. If `n_blocks_per_location` is given it must be a multiple of the number of replicates. Designs from the algebraic methods are always resolvable.

`n_candidates` Maximum number of candidate designs (`"search"`) or of random starting arrays (`"alpha"`, `"alpha2"`) to generate. `summary$n_candidates_evaluated` reports how many were used.

`seed` Optional random seed.

`field_cols` Optional number of field columns for row/column layout. Can be a single integer or a named vector indexed by location.

`rows_per_block` Optional number of physical field rows each incomplete block should occupy. Can be a single integer or a named vector for selected locations. Must be used with `blocks_per_row` for those locations.

`blocks_per_row` Optional number of incomplete blocks to place side by side in each field row band. Can be a single integer or a named vector for selected locations. Must be used with `rows_per_block` for those locations.

`serpentine` Logical. If `TRUE`, row/column coordinates use serpentine ordering within each location.

`output_file` Optional tab-delimited output file for the complete design.

`method` Construction method: `"search"`, `"alpha"` or `"alpha2"` (see Details).

`n_prep_per_block`
For the algebraic methods, the number `m` of p-rep entries in every block. If `NULL`, the largest feasible `m` whose blocks fit in `block_size` is used.

`n_efficiency_candidates`
Number of optimal candidates (no entry repeated within a block, connected locations, and no pair of entries sharing more than one block) to compare on the average efficiency factor; the one with the highest efficiency is returned. Use `1` to return the first optimal candidate. Efficiency is compared for up to 2000 entries.

Details

Three construction methods are available through `method`:

`"search"` **(default)** A flexible randomized search. Any number of entries, locations, block sizes, p-rep proportions (`prep_prop`, `n_duplicate_per_location`), replication levels (`replications_per_repeated_entry`) and block counts can be used, with an optional resolvable layout (`resolvable`).

`"alpha"` The partial alpha-array construction of Section 2.1 of the paper. An alpha-array for `l * m` rows and `2l` replicates is searched, the `m x 2` rectangles on its diagonal give the two-replicate design for the p-rep entries of each location, and half of the remaining elements of each column are deleted (Section 2.2) so that the design is a (0,1)-design. Every entry is duplicated at exactly one location, each location is resolvable with two replicates of `s` blocks, and each block holds `m` p-rep entries; the final block size is `(l + 1) m / 2` (blocks of the two replicates differ by one plot when this is not an integer).

`"alpha2"` The augmented alpha(2)-array construction of Section 2.4, for even `m` and an odd number of blocks per replicate `s`. Every block has `(l + 1) m / 2` plots.

For the algebraic methods, `m` is set with `n_prep_per_block` (or chosen as the largest value whose blocks fit in `block_size`), the number of blocks and of duplicated entries follow from the construction, and `replications_per_repeated_entry` must be 2. When the number of entries is not a multiple of the array size, the surplus array positions are dropped as dummy entries, so some blocks are one plot smaller. `augmented_prep_parameters()` lists the feasible parameter sets.

The incomplete block structure is independent from the field geometry. By default, `field_cols` lays the ordered plots into rows and columns. For a rectangular block layout, use `rows_per_block` and `blocks_per_row` to place multiple blocks side by side within each location.

With `method = "search"`, the p-rep plots of every location are spread evenly over its blocks, and the p-rep entries are placed so that they link all blocks of the location (the location is connected), so entry effects can also be estimated from each location on its own. A warning is given when there are too few p-rep plots to connect the blocks. After the random greedy search, the best candidates are improved by swapping plots between blocks to remove pairs of entries that share more than one block.

For all methods, `summary` reports the maximum pairwise concurrence, the number of locations whose blocks are not connected, and the average efficiency factor `efficiency_factor` of the block design (Section 2.3 of the paper; higher is better), which can be used to compare designs. Entry labels, block order within replicates and plot order within blocks are randomized.

Value

A list with design inputs, summary diagnostics, complete plot-level design, Breedbase-compatible design columns, block summary, replication summary, repeated-entry groups, and pairwise concurrence matrix. In the complete design, `repeated_at` is the location where the entry receives its extra p-rep plot(s), or `"not_repeated"` when the entry is not selected for extra replication. `replicate` is the replicate (superblock) of the plot for resolvable designs, and 1 otherwise; the Breedbase `rep` column is `replicate` for resolvable designs and the plot's occurrence number otherwise.

References

Williams, E., Piepho, H.-P. and Whitaker, D. (2011). Augmented p-rep designs. Biometrical Journal 53, 19-27. doi:10.1002/bimj.201000102

See Also

`augmented_prep_parameters()`, `analyze_augmented_prep_design()`, `plot_augmented_prep_design()`

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2", "Loc3"),
  block_size = c(Loc1 = 5, Loc2 = 5, Loc3 = 6),
  n_blocks_per_location = c(Loc1 = 4, Loc2 = 4, Loc3 = 4),
  n_duplicate_per_location = c(Loc1 = 4, Loc2 = 2, Loc3 = 4),
  replications_per_repeated_entry = c(Loc1 = 2, Loc2 = 3, Loc3 = 2),
```

```
  n_candidates = 5,
  seed = 123,
  field_cols = 8
)

design$summary
head(design$design)

# Example 1 of Williams, Piepho and Whitaker (2011): 20 entries, two
# locations, blocks of 3 built from a partial alpha-array.
alpha <- augmented_prep_design(
  treatments = paste0("G", 1:20),
  locations = c("L1", "L2"),
  method = "alpha",
  block_size = 3,
  seed = 1
)
alpha$summary[, c("method", "m", "s", "max_pairwise_concurrence", "efficiency_factor")]
```

`augmented_prep_parameters` *Feasible parameters for augmented p-rep constructions*

Description

Lists the parameter sets that `augmented_prep_design()` can build for a given number of entries and locations, in the spirit of Table 3 of Williams, Piepho and Whitaker (2011). For each number `m` of p-rep entries per block, it gives the number of blocks per replicate `s`, the final block size(s), the number of entries duplicated at each location, and how many "dummy" entries are dropped when the number of entries is not a multiple of the array size (dropping them makes some blocks one plot smaller).

Usage

```
augmented_prep_parameters(n_treatments, n_locations, max_block_size = 20)
```

Arguments

`n_treatments`	Number of entries (`v`).
`n_locations`	Number of locations (`l`).
`max_block_size`	Largest final block size to list.

Value

A tibble with one row per method and `m`: `method`, `m`, `s`, `block_size` (largest final block), `min_block_size`, `blocks_per_location`, `plots_per_location` (largest), `duplicated_per_location` (largest), `dummies`, `feasible`, and `note`.

References

Williams, E., Piepho, H.-P. and Whitaker, D. (2011). Augmented p-rep designs. Biometrical Journal 53, 19-27. doi:10.1002/bimj.201000102

Examples

```
augmented_prep_parameters(n_treatments = 120, n_locations = 6, max_block_size = 10)
```

`augmented_row_column_capacity`
Calculate augmented row-column design capacity

Description

Reports the plot counts, error degrees of freedom and feasibility of an augmented row-column layout with superimposed blocks (Piepho and Williams, 2016). Rows are grouped into row groups and columns into column groups; their intersections are the blocks. Groups can have equal sizes (`rows_per_block`, `cols_per_block`) or unequal sizes (`rows_per_rowgroup`, `cols_per_colgroup`), and blocks can hold any number of check plots (`checks_per_block`).

Usage

```
augmented_row_column_capacity(
  rows_in_field = NULL,
  cols_in_field = NULL,
  rows_per_block = NULL,
  cols_per_block = NULL,
  n_controls,
  rows_per_rowgroup = NULL,
  cols_per_colgroup = NULL,
  checks_per_block = NULL
)
```

Arguments

`rows_in_field`	Number of rows in the whole field.
`cols_in_field`	Number of columns in the whole field.
`rows_per_block`	Number of rows in each row group (equal groups).
`cols_per_block`	Number of columns in each column group (equal groups).
`n_controls`	Number of replicated checks/controls (`vc`).
`rows_per_rowgroup`	Optional vector with the number of rows of each row group (`kb(i)`), for unequal groups. Replaces `rows_per_block`.

`cols_per_colgroup`
: Optional vector with the number of columns of each column group (`sb(j)`). Replaces `cols_per_block`.

`checks_per_block`
: Optional number of check plots per block (`pb(i, j)`): a single number, a matrix with one row per row group and one column per column group, or a vector by row group. Defaults to `n_controls` (every block a complete block of checks). Smaller values give incomplete blocks; larger values give extra check replication.

Value

A one-row tibble with the field size, number of groups and blocks, check and entry plots, `error_df`, the error degrees of freedom of the row-column ANOVA, `check_plots - n_controls - (rows - 1) - (cols - 1) - (row` (a design is only analyzable when `error_df > 0`; Clarke and Stefanova (2011) recommend about 50), `check_fraction`, and the smallest slack of the row-group and column-group conditions of Piepho and Williams (2016), which must be non-negative for the row-group and column-group sub-designs to be estimable on their own.

References

Piepho, H.-P. and Williams, E. R. (2016). Augmented row-column designs for a small number of checks. Agronomy Journal 108, 2256-2262. doi:10.2134/agronj2016.06.0325

Examples

```
augmented_row_column_capacity(
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_controls = 4
)

# Example 3 of Piepho and Williams (2016): unequal groups and blocks.
augmented_row_column_capacity(
  n_controls = 4,
  rows_per_rowgroup = c(6, 5, 6, 5),
  cols_per_colgroup = c(3, 4, 3),
  checks_per_block = rbind(c(4, 5, 4), c(3, 5, 3), c(3, 5, 4), c(3, 5, 3))
)
```

`augmented_row_column_design` *Generate an augmented row-column field design*

Description

Randomizes unreplicated treatments and replicated controls (checks) in an augmented row-column design with superimposed blocks (Piepho and Williams, 2016). Adjacent rows form row groups and adjacent columns form column groups; each intersection is a block that receives a given number of check plots. Row, column and block effects are all estimable from the checks, and the layout must leave positive error degrees of freedom (see `augmented_row_column_capacity()`).

Usage

```
augmented_row_column_design(
  treatments,
  controls,
  rows_in_field = NULL,
  cols_in_field = NULL,
  rows_per_block = NULL,
  cols_per_block = NULL,
  plot_type = c("serpentine", "cartesian"),
  n_candidates = 1000,
  seed = NULL,
  output_file = NULL,
  rows_per_rowgroup = NULL,
  cols_per_colgroup = NULL,
  checks_per_block = NULL,
  method = c("auto", "latin", "optimal")
)
```

Arguments

`treatments`	Character vector of unreplicated treatment/entry names. Its length must equal the number of entry plots.
`controls`	Character vector of replicated check/control names.
`rows_in_field`	Number of rows in the whole field.
`cols_in_field`	Number of columns in the whole field.
`rows_per_block`	Number of rows in each row group (equal groups).
`cols_per_block`	Number of columns in each column group (equal groups).
`plot_type`	Plot numbering order. Use `"serpentine"` or `"cartesian"`.
`n_candidates`	Number of candidate layouts (`"latin"`) or random restarts of the local search (`"optimal"`, at most 50; the search stops after 3 restarts without improvement).
`seed`	Optional random seed.
`output_file`	Optional path for writing the complete design as a tab-delimited file.
`rows_per_rowgroup`, `cols_per_colgroup`	Optional vectors of unequal group sizes, as in `augmented_row_column_capacity()`.
`checks_per_block`	Optional number of check plots per block, as in `augmented_row_column_capacity()`.
`method`	Check allocation method: `"auto"`, `"latin"` or `"optimal"`.

Details

Two check-allocation methods are available:

`"latin"` For equal groups where every block is a complete block of checks (one copy of each). Checks are placed with covering Latin rectangles, so controls never share a row or column within a block, the same control never appears twice in a field row or column, and every field row and column has a check. Candidates are scored for spatial spread and row/column balance.

`"optimal"` The general strategy of Piepho and Williams (2016): unequal group sizes and any number of check plots per block (incomplete blocks or extra replication of checks). Check labels are first allocated to blocks balancing row groups, column groups and blocks (Step 2); check plots are then placed within blocks and the layout improved by a local search that maximizes the D-criterion of the check-plot model `row + column + rowgroup:colgroup + check` (Steps 3-5). Check plots next to each other (same column in consecutive rows, or same row in consecutive columns) are avoided, inside blocks above all, and spatial spread breaks ties. Entries fill the remaining plots at random (Step 6).

`method = "auto"` uses `"latin"` when it applies and `"optimal"` otherwise.

Value

A list with:

input Validated design inputs.

summary Design size, error df, method, and the D-criterion `d_criterion` (`det(X'X)^(1/p)` of the check-plot model; higher is better), and the number of pairs of neighbouring check plots (same column in consecutive rows, or same row in consecutive columns) inside a block and across block boundaries.

design Complete plot-level design with row, column, block, entry, control flag, and plot order.

breedbase_design Breedbase-compatible columns: `plots`, `block`, `all_entries`, `rep`, and `is_control`.

References

Piepho, H.-P. and Williams, E. R. (2016). Augmented row-column designs for a small number of checks. Agronomy Journal 108, 2256-2262. doi:10.2134/agronj2016.06.0325

Examples

```
treatments <- paste0("G", seq_len(108))
controls <- paste0("Check", seq_len(4))

design <- augmented_row_column_design(
  treatments = treatments,
  controls = controls,
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 50,
```

```
  seed = 123
)

head(design$design)
design$summary

# Example 3 of Piepho and Williams (2016): 173 entries, 4 checks, unequal
# row and column groups, and 3 to 5 check plots per block.
pb <- rbind(c(4, 5, 4), c(3, 5, 3), c(3, 5, 4), c(3, 5, 3))
ex3 <- augmented_row_column_design(
  treatments = paste0("E", 1:173),
  controls = c("A", "B", "C", "D"),
  rows_per_rowgroup = c(6, 5, 6, 5),
  cols_per_colgroup = c(3, 4, 3),
  checks_per_block = pb,
  n_candidates = 3,
  seed = 1
)
ex3$summary[, c("method", "check_plots", "entry_plots", "error_df", "d_criterion")]
```

`augmented_row_column_layouts` *List the analyzable layouts of an augmented row-column field*

Description

Tries every combination of block size and number of checks for a field and returns the layouts that can be analysed, so a layout can be chosen without writing a loop over `augmented_row_column_capacity()`. Each block gets one plot of every check.

Usage

```
augmented_row_column_layouts(
  rows_in_field,
  cols_in_field,
  rows_per_block = NULL,
  cols_per_block = NULL,
  n_controls = 2:6,
  min_error_df = 1,
  n_entries = NULL
)
```

Arguments

`rows_in_field, cols_in_field`
: Field size.

`rows_per_block, cols_per_block`
: Block sizes to try. By default every divisor of the field size from 2 up to the field size.

`n_controls`
: Numbers of checks to try.

`min_error_df`
: Smallest error df to keep.

`n_entries`
: Optional number of entries to plant. Only layouts with at least that many entry plots are kept, and `spare_plots` shows how many entry plots would be left over.

Details

There is a trade-off: more check plots mean fewer entries but more error degrees of freedom, i.e. more reliable F-tests and spatial adjustment. Aim for at least about 10 error df when the field allows it (Clarke and Stefanova, 2011, recommend about 50 for large trials).

Value

A tibble with one row per analyzable layout, sorted by entry plots (most first) and then error df: block sizes, number of checks, blocks, entry and check plots, `check_fraction`, `error_df`, `method` (the construction that `augmented_row_column_design()` uses for it: `"latin"` or `"optimal"`) and, when `n_entries` is given, `spare_plots`.

See Also

`augmented_row_column_capacity()`, `augmented_row_column_design()`

Examples

```
# All analyzable layouts of a 12 x 12 field with 2 to 5 checks:
augmented_row_column_layouts(12, 12, n_controls = 2:5)

# Layouts that fit 100 entries with at least 10 error df:
augmented_row_column_layouts(12, 12, n_controls = 2:6, min_error_df = 10, n_entries = 100)
```

`breedbase_adjusted_trait` *Find or create the Breedbase trait for adjusted values*

Description

First-stage values need their own trait, so raw and adjusted values stay apart. Breedbase makes such traits by composing the measured trait with a statistic term, e.g. "fresh root yield" + "Adjusted Means from Spatial Correction using SpATS R" gives the composed trait `"fresh root yield|Adjusted Means from Spatial Correction using SpATS R"` (ontology `COMP`). This function finds that trait, or creates it with `create = TRUE` through the same calls as the "Compose a new trait" page of Breedbase.

Usage

```
breedbase_adjusted_trait(
  con,
  traits,
  statistic = "Adjusted Means from Spatial Correction using SpATS R",
  create = FALSE
)
```

Arguments

con	A connection from breedbase_login().
traits	Measured trait names, as in Breedbase.
statistic	The statistic term (from the SGNSTAT ontology of Breedbase), with or without its id. The default is the term of the Breedbase spatial correction tool; for non-spatial models "Adjusted means from BLUEs using LMER R" is another choice.
create	Logical. Create the traits that do not exist yet.

Value

A tibble with trait, adjusted_trait, adjusted_trait_db_id and status ("exists", "created" or "missing").

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
breedbase_adjusted_trait(con, "fresh root yield")                  # look only
breedbase_adjusted_trait(con, "fresh root yield", create = TRUE)   # create if needed

## End(Not run)
```

---

breedbase_check *Check names in Breedbase before building a design*

Description

Run this first, before designing the trial: it checks that the location(s) and every entry and check are already in Breedbase (and, when given, the breeding program, the folder and that the trial name is free). All problems are reported together, with close matches as suggestions (for example "ibadan" -> "Ibadan"), so they can be fixed in one go. save_breedbase() repeats the same checks before writing.

Usage

```
breedbase_check(
  con,
  accessions = NULL,
  location = NULL,
  program = NULL,
  folder = NULL,
  trial_name = NULL
)
```

Arguments

con	A connection from breedbase_login().
accessions	Entry and check names: a character vector, or a design from augmented_row_column_design(), augmented_prep_design() or resolvable_row_column_design().
location	Breedbase location name(s).
program, folder	Optional breeding program and folder (the folder is optional for saving too: without it the trial goes directly in the program).
trial_name	Optional trial name. A trial with this name that already exists (with plots) is reported to fix.

Value

Invisibly, a tibble with one row per name checked: what, name, status ("OK", "MISSING", "NEW", "TAKEN", "EMPTY" or "REPEATED") and suggestion. Its attribute "ok" is TRUE when nothing needs fixing.

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so save_breedbase() after breedbase_check() does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run save_breedbase() again: an existing folder is used, and a trial left without plots gets its plots. Settings: options(breedtools.write_wait = 30, breedtools.write_checks = 10). Settings: options(breedtools.germplasm_batch = 10) for smaller batches, options(breedtools.breedbase_timeout = 600) to wait longer (seconds) for each answer.

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
entries <- c(paste0("IITA-TMS-IBA", 1:108))
```

```
checks <- c("TMEB419", "IITA-TMS-IBA30572", "TMEB693", "IITA-TMS-IBA000070")
breedbase_check(con, accessions = c(entries, checks), location = "Ibadan",
                program = "IITA", folder = "AYT 2026", trial_name = "AYT_2026_Ibadan")

## End(Not run)
```

`breedbase_lists` *List breeding programs, folders and locations in Breedbase*

Description

Use these to find the exact names to give to `save_breedbase()`.

Usage

```
breedbase_programs(con)

breedbase_folders(con, program)

breedbase_locations(con)
```

Arguments

`con`	A connection from `breedbase_login()`.
`program`	For `breedbase_folders()`: the breeding program name.

Value

A tibble with names and database ids.

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
breedbase_programs(con)
breedbase_folders(con, program = "IITA")
breedbase_locations(con)

## End(Not run)
```

`breedbase_login` *Log in to a Breedbase server*

Description

Logs in through the BrAPI token call and returns a connection used by the other `breedbase_*()` functions and by `save_breedbase()`. Only the access token is kept; the password is not stored.

Usage

```
breedbase_login(url, username, password = NULL)
```

Arguments

`url`	Address of the Breedbase site, e.g. `"https://cassavabase.org"`.
`username`	Your Breedbase username.
`password`	Optional password; see Details.

Details

Never write your password in a script. Leave `password` empty: it is read from the environment variable `BREEDBASE_PASSWORD` if set (for example in your `.Renviron` file), and otherwise you are asked for it in a pop-up.

Value

A `breedbase_connection` object.

See Also

`save_breedbase()`, `breedbase_programs()`, `breedbase_folders()`, `breedbase_locations()`

Examples

```
## Not run:
con <- breedbase_login("https://cassavabase.org", username = "jdoe")
con

## End(Not run)
```

calculate_environment_stratification_anova *Calculate design-aware ANOVA*

Description

Calculate design-aware ANOVA

Usage

```
calculate_environment_stratification_anova(data)
```

Arguments

data	Standardized phenotype data.

Value

ANOVA table as a data frame.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

calculate_environment_stratification_anova(prepared$data)
```

check_design *Standard checks of a design*

Description

Runs the structural checks that every design of its kind should pass and prints one [OK] / [FAIL] line per check ([NOTE] for informational ones). The results are recorded for check_summary().

Usage

```
check_design(design, section = NULL)
```

Arguments

design	A design object from augmented_row_column_design(), augmented_prep_design() or resolvable_row_column_design().
section	Optional label for the recorded checks. Defaults to the label set with check_section(), or else the kind of design.

Details

Augmented row-column designs every plot used once, every entry planted once, every field row and column has a check, the requested number of check plots in each block, positive error df, all row, column and block effects estimable from the checks, and no check plots next to each other inside a block (a note for neighbours across block boundaries).

Augmented p-rep designs every entry at every location, no entry twice in a block, block sizes within a location differ by at most one, every location connected, the pairwise concurrence (a note when pairs share more than one block), and the replicate structure of resolvable designs.

Resolvable row-column designs every treatment once in every replicate, every plot used once, the spatial efficiency below its upper bound, and latinization when requested.

Value

Invisibly, TRUE if all (non-note) checks passed.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
check_reset()
check_design(rc)
check_summary()
check_reset()
```

---

check_reset *Clear the recorded checks*

Description

Removes the checks recorded so far with check_result() and check_design(), so that the next check_summary() only counts new checks. It does not check or change any design or data; it only empties the list. A message says how many earlier checks were cleared and how many of them had passed.

Usage

```
check_reset()
```

Value

Invisibly, a tibble of the checks that were cleared.

Examples

```
check_reset()
check_result("one equals one", 1 == 1)
check_reset()
```

`check_result` *Record and print a check*

Description

Prints `[OK]` or `[FAIL]` followed by a description, and records the result so that `check_summary()` can report how many checks passed. Use it to write your own checks while exploring a design or an analysis.

Usage

```
check_result(what, ok, section = NULL, note = FALSE)
```

Arguments

`what`	Description of the check, e.g. `"every entry planted once"`.
`ok`	Logical. `TRUE` when the check passes; anything else (including `NA`) counts as a failure.
`section`	Optional label for grouping checks, e.g. `"p-rep"`. Defaults to the label set with `check_section()`.
`note`	Logical. If `TRUE`, the check is informational: it prints `[NOTE]` instead of `[FAIL]` when ok is not `TRUE` and is not counted as a failure.

Value

Invisibly, `TRUE` if the check passed.

See Also

`check_section()`, `check_summary()`, `check_reset()`, `check_design()`

Examples

```
check_reset()
check_result("two plus two is four", 2 + 2 == 4)
check_result("a deliberate failure", FALSE, section = "demo")
check_summary()
check_reset()
```

check_section *Set the section label for the next checks*

Description

Checks recorded after this call with `check_result()` and `check_design()` are grouped under `name` in `check_summary()`.

Usage

```
check_section(name)
```

Arguments

`name`	Section label, e.g. `"1. Row-column design"`. `NULL` removes the label.

Value

Invisibly, the previous label.

Examples

```
check_reset()
check_section("arithmetic")
check_result("one equals one", 1 == 1)
check_summary()
check_reset()
```

check_setup *Check that everything is ready before starting*

Description

Run this at the start of a script. It starts a fresh list of checks (so the final `check_summary()` only counts this run) and checks that you are ready to design and analyse trials:

- breedtools is loaded, and which version;
- the optional packages 'lme4' (mixed models with random blocks, random entries or random genotype-by-location) and 'SpATS' (spatial models) are installed; a missing one is a note, not a failure, and the message says what it is needed for;
- the folder for your files exists (it is created if needed) and files can be written to it.

Usage

```
check_setup(out_dir = NULL)
```

Arguments

out_dir	Optional folder where the script will write field books and other files.

Value

Invisibly, TRUE if everything needed is in place.

See Also

check_summary(), check_design(), check_result()

Examples

```
check_setup(out_dir = file.path(tempdir(), "my_trials"))
```

---

check_summary *Summary of the recorded checks*

Description

Reports what was checked and what is correct: how many of the checks recorded with check_result() and check_design() passed, broken down by section, followed by the checks that failed and the notes.

Usage

```
check_summary(details = FALSE, print = TRUE)
```

Arguments

details	Logical. If TRUE, also list every check with its status.
print	Logical. If TRUE (default), print the report.

Value

Invisibly, a tibble of all recorded checks with columns section, check and status ("OK", "FAIL" or "NOTE").

Examples

```
check_reset()
check_result("one equals one", 1 == 1, section = "arithmetic")
check_result("two is even", 2 %% 2 == 0, section = "arithmetic")
check_summary()
check_summary(details = TRUE)
check_reset()
```

complete_environment_info *Complete environment metadata*

Description

Complete environment metadata

Usage

```
complete_environment_info(env_info)
```

Arguments

env_info	Data frame with environment, location, trial, and year.

Value

Data frame with an added `environment_label` column.

Examples

```
env_info <- data.frame(
  environment = c("Loc1_T1", "Loc2_T2"),
  location = c("Loc1", "Loc2"),
  trial = c("T1", "T2"),
  year = c("2025", NA)
)

complete_environment_info(env_info)
```

detect_design *Detect field trial design for environment stratification*

Description

Detects whether the data look like CRD, RCBD, row-column, alpha-lattice, or augmented/block-only design.

Usage

```
detect_design(data)
```

Arguments

data	Standardized phenotype data.

Value

A list with `design`, `formula`, and `message`.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

design <- detect_design(prepared$data)
design$design
design$formula
```

---

environment_summary_by_group

Summarize environment names by group

Description

Summarize environment names by group

Usage

```
environment_summary_by_group(group_summary, group_membership)
```

Arguments

`group_summary`	Group summary table.
`group_membership`	Group membership table.

Value

Group summary with display metadata.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

lin <- lin_group_environments(prepared$data)
membership <- add_environment_metadata(lin$group_membership, prepared$env_info)
environment_summary_by_group(lin$group_summary, membership)
```

`first_stage_values` *First-stage plot values from an analysis*

Description

Turns analyses of a trial into one value per field plot, ready for `save_breedbase_first_stage()`: the genotype's adjusted mean plus the plot's residual. This removes the design effects (blocks, rows, columns, replicates) and the spatial trend, and keeps the plot-to-plot variation, so the result has as many plots as the field trial. With SpATS (`spatial = TRUE` or `stage1 = "spatial"`) these are spatially corrected plot values.

Usage

```
first_stage_values(
  fits,
  design,
  trial_name,
  trait = NULL,
  plot_name_pattern = "{trial}_{accession}_{block}_{rep}",
  plot_names = NULL
)
```

Arguments

`fits`	One analysis, or a named list of analyses, one per trait (the names are the Breedbase trait names). Analyses from `analyze_augmented_row_column_design()`, `analyze_augmented_prep_design()`, `analyze_augmented_prep_two_stage()` (stage-1 fits per location) or `analyze_resolvable_row_column()`.
`design`	The design that was saved to Breedbase.
`trial_name`	The trial name used when saving the design.
`trait`	Trait name when `fits` is a single analysis.
`plot_name_pattern, plot_names`	The plot naming used in `save_breedbase()`.

Value

A tibble with `plot_name`, `trial_name`, `accession`, `plot_number`, `block`, `rep`, `row`, `col`, `is_check` and one column per trait. Its attribute `"method"` is `"spatial"` when any analysis used SpATS, otherwise `"mixed"`.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
set.seed(1)
d$yield <- 25 + 0.4 * d$row + rnorm(nrow(d), sd = 2)
```

```
fit <- analyze_augmented_row_column_design(d, "yield")
values <- first_stage_values(fit, rc, "AYT_2026_Ibadan", trait = "fresh root yield")
head(values)
```

`get_compatible_groups` *Get compatible environment groups*

Description

Get compatible environment groups

Usage

```
get_compatible_groups(group_tests)
```

Arguments

`group_tests`	Output from `make_environment_groups()`.

Value

A tibble containing only homogeneous groups.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 30),
  df_error = c(20, 20, 20)
)

group_tests <- make_environment_groups(env_anova, n_sim = 2000)
get_compatible_groups(group_tests)
```

`get_maximal_groups` *Select maximal compatible environment groups*

Description

Removes smaller compatible groups that are fully contained inside larger compatible groups.

Usage

```
get_maximal_groups(compatible_groups)
```

Arguments

compatible_groups
: Output from get_compatible_groups().

Value

A tibble with maximal compatible groups only.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 30),
  df_error = c(20, 20, 20)
)

group_tests <- make_environment_groups(env_anova, n_sim = 2000)
get_maximal_groups(get_compatible_groups(group_tests))
```

---

guess_column *Guess a column name from possible Breedbase column names*

Description

Guess a column name from possible Breedbase column names

Usage

```
guess_column(df, possible_names)
```

Arguments

df
: A data frame.

possible_names
: A character vector of possible column names.

Value

A column name if found, otherwise NULL.

Examples

```
df <- data.frame(germplasmName = "G1", phenotype = 10)
guess_column(df, c("germplasmName", "accession_name"))
```

hartley_critical_sim *Simulate Hartley Fmax critical value*

Description

Estimates the Hartley Fmax critical value using simulation. This version can handle different residual degrees of freedom across environments.

Usage

```
hartley_critical_sim(df_error_vec, alpha = 0.05, n_sim = 1e+05, seed = 123)
```

Arguments

df_error_vec	Numeric vector of residual degrees of freedom.
alpha	Significance level. Default is 0.05.
n_sim	Number of simulations. Default is 100000.
seed	Random seed used for the simulation. The user's global random number generator state is restored on exit. Use NULL to draw from the current random number stream without setting a seed.

Value

A numeric critical value.

Examples

```
hartley_critical_sim(c(20, 20, 20), alpha = 0.05, n_sim = 2000)
```

hartley_fmax_test *Hartley Fmax test for residual variance homogeneity*

Description

Hartley Fmax test for residual variance homogeneity

Usage

```
hartley_fmax_test(env_anova, alpha = 0.05, n_sim = 1e+05, seed = 123)
```

Arguments

env_anova	A data frame containing environment-level ANOVA results. It must include columns environment, ms_error, and df_error.
alpha	Significance level. Default is 0.05.
n_sim	Number of simulations for the critical value.
seed	Random seed passed to hartley_critical_sim(). The user's global random number generator state is not modified.

Value

A tibble with observed Fmax, critical Fmax, and homogeneity decision.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 15),
  df_error = c(20, 20, 20)
)

hartley_fmax_test(env_anova, n_sim = 2000)
```

---

lin_error_mse *Calculate residual error MSE for a Lin environment group*

Description

Calculate residual error MSE for a Lin environment group

Usage

```
lin_error_mse(data, envs, common_genotypes)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments.
common_genotypes	Character vector of common genotypes.

Value

A list with MSE, residual df, and message.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

envs <- prepared$env_info$environment[1:2]
genotypes <- unique(prepared$data$accession_name)
lin_error_mse(prepared$data, envs, genotypes)
```

---

lin_group_environments

Group environments using Lin-style GE compatibility testing

Description

Group environments using Lin-style GE compatibility testing

Usage

```
lin_group_environments(data, alpha = 0.05)
```

Arguments

data	Standardized phenotype data.
alpha	Significance level.

Value

A list with pairwise tests, group summary, group membership, and ungrouped environments.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

lin <- lin_group_environments(prepared$data, alpha = 0.05)
lin$group_summary
lin$group_membership
```

lin_ss_ge *Calculate Lin genotype-by-environment sum of squares*

Description

Calculate Lin genotype-by-environment sum of squares

Usage

```
lin_ss_ge(data, envs)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments to test.

Value

A list with a summary table and common genotype names.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

envs <- prepared$env_info$environment[1:2]
lin_ss_ge(prepared$data, envs)$summary
```

lin_test_group *Test one Lin environment group*

Description

Test one Lin environment group

Usage

```
lin_test_group(data, envs, alpha = 0.05)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments.
alpha	Significance level.

Value

Data frame with Lin GE test result.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

envs <- prepared$env_info$environment[1:2]
lin_test_group(prepared$data, envs, alpha = 0.05)
```

---

make_environment_groups

Test all compatible environment groups using Hartley Fmax

Description

Tests all combinations of environments and identifies groups with homogeneous residual variances.

Usage

```
make_environment_groups(
  env_anova,
  alpha = 0.05,
  max_group_size = Inf,
  n_sim = 1e+05,
  seed = 123
)
```

Arguments

env_anova	A data frame from analyze_environment_anova().
alpha	Significance level.
max_group_size	Maximum environment group size to test.
n_sim	Number of simulations for Hartley critical value.
seed	Random seed.

Value

A tibble with all tested groups.

Examples

```
env_anova <- data.frame(
  environment = c("Loc1", "Loc2", "Loc3"),
  ms_error = c(10, 12, 30),
  df_error = c(20, 20, 20)
)

make_environment_groups(env_anova, n_sim = 2000)
```

`map_environment_groups` *Create a map of environment groups*

Description

Creates an interactive `leaflet` map where locations belonging to the same environment group use the same marker color. Ungrouped environments are shown with `ungrouped_color`.

Usage

```
map_environment_groups(
  stratification_result,
  locations,
  ignore_case = FALSE,
  group_palette = NULL,
  ungrouped_color = "#7A7A7A",
  match_by = "location",
  strict = TRUE,
  tiles = "breedbase"
)
```

Arguments

`stratification_result`	Result from `run_environment_stratification()` or `run_breedbase_environment_stratificatior`
`locations`	Location coordinates from `read_locations()`, a location file path, or a data frame with location, latitude, and longitude columns.
`ignore_case`	Logical. If `TRUE`, location matching ignores case.
`group_palette`	Optional vector of colors used for environment groups.
`ungrouped_color`	Color for ungrouped environments.
`match_by`	Name from the stratification result used to match the `location` column in `locations`. Use `"location"` when coordinates are named by location, `"environment"` when coordinates are named by the constructed environment identifier, or `"environment_label"` when coordinates are named by display labels.

strict — Logical. If TRUE, stop when any environment location has no matching coordinates.

tiles — Background map. The default, "breedbase", uses the same maps as the 'Breedbase' Locations page: Esri Street (shown first), Topographical and Satellite, with a switch in the top-right corner. They need no account or API key and work in the RStudio viewer and in saved HTML files. "street", "topo" or "satellite" shows only that map. "OpenStreetMap" uses OpenStreetMap's own tiles (these may be refused outside a website). Any other value is passed to leaflet::addProviderTiles() as a provider name; some commercial providers need an API key.

Value

A leaflet htmlwidget.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
loc_file <- system.file("extdata", "breedbase_locations.csv", package = "breedtools")

result <- run_environment_stratification(pheno_file, trait = "fresh root yield")
locations <- read_locations(loc_file, phenotype_data = result)

group_map <- map_environment_groups(result, locations)
if (interactive()) group_map
```

---

phenotype_template *Phenotyping template with the Breedbase plot names of a design*

Description

Makes a table with one row per plot, the plot names that save_breedbase() gives (or gave) to the plots, the design columns (accession, block, rep, row, column) and one empty column per trait. Fill it in (in R or in a spreadsheet) and upload it with save_breedbase_phenotypes().

Usage

```
phenotype_template(
  design,
  trial_name,
  traits = character(0),
  plot_name_pattern = "{trial}_{accession}_{block}_{rep}",
  plot_names = NULL
)
```

Arguments

design	A design from augmented_row_column_design(), augmented_prep_design() or resolvable_row_column_design().
trial_name	The trial name used when saving the design. For p-rep designs saved as one trial per location, the location is added as in save_breedbase().
traits	Trait names as in Breedbase, e.g. "fresh root yield" or "fresh root yield|CO_334:0000013". Can be empty (no trait columns).
plot_name_pattern, plot_names	The same plot naming used in save_breedbase().

Value

A tibble: plot_name, trial_name, accession, plot_number, block, rep, row, col, is_check and one column per trait.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
sheet <- phenotype_template(rc, "AYT_2026_Ibadan",
                        traits = c("fresh root yield", "dry matter content percentage"))
head(sheet)
```

---

plot_augmented_prep_design

Plot an augmented p-rep design grid

Description

Draws one field map per location for an augmented p-rep design.

Usage

```
plot_augmented_prep_design(
  design,
  locations = NULL,
  label = c("all_entries", "entry_type", "repeated_at", "block", "replicate", "none"),
  fill = c("entry_type", "is_extra_repeat", "repeated_at", "block", "replicate"),
  show_plot_labels = TRUE,
  show_block_labels = FALSE,
  block_label_position = c("background", "center"),
  label_cex = NULL,
  block_label_cex = NULL,
  block_label_col = NULL,
  block_lwd = 2,
```

```
  main = NULL,
  legend = TRUE,
  caption = TRUE,
  ...
)
```

Arguments

design	An object returned by augmented_prep_design() or its design data frame.
locations	Optional character vector of locations to plot. Defaults to all locations in design order.
label	Cell label to show. Use "all_entries", "entry_type", "repeated_at", "block", "replicate", or "none".
fill	Cell coloring variable. Use "entry_type", "is_extra_repeat", "repeated_at", "block", or "replicate".
show_plot_labels	Logical. If TRUE, labels are printed inside cells unless label = "none".
show_block_labels	Logical. If TRUE, block numbers are printed. Defaults to FALSE so block labels do not overlap accession labels.
block_label_position	Position for block labels when show_block_labels = TRUE. Use "background" for a faint watermark behind accession labels or "center" for a regular center label.
label_cex	Optional text size for plot-cell labels.
block_label_cex	Optional text size for block labels.
block_label_col	Color for block labels.
block_lwd	Line width for block boundaries.
main	Optional plot title. If multiple locations are plotted, a single value is combined with the location name; a vector matching the number of locations is used panel by panel.
legend	Logical. If TRUE, draw a legend for cell colors.
caption	Logical. If TRUE, print a short explanation of the colours and lines under each map.
...	Additional arguments passed to graphics::plot.window().

Details

A p-rep design has no check plots: at each location some entries are planted more than once (repeated entries), and these replace the checks for estimating the error and the block effects. By default the map colours the repeated entries (orange, names in bold) and the entries with a single plot at that location (light grey). Thick black lines outline the incomplete blocks; in resolvable designs a thick blue line separates the replicates, which are labelled at the right. A caption under each map explains this.

Value

Invisibly returns the plotted design data frame with internal plotting coordinates.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2", "Loc3"),
  block_size = 5,
  n_candidates = 5,
  seed = 123,
  field_cols = 5
)

plot_augmented_prep_design(design)
```

---

plot_augmented_row_column_design

Plot an augmented row-column design grid

Description

Draws a field grid for an augmented row-column design, with plot cells, row and column labels, and thick outlines around row-column blocks.

Usage

```
plot_augmented_row_column_design(
  design,
  label = c("all_entries", "type", "block", "none"),
  fill = c("type", "block", "rowgroup", "colgroup", "is_control"),
  show_plot_labels = TRUE,
  show_block_labels = FALSE,
  block_label_position = c("background", "center"),
  label_cex = NULL,
  block_label_cex = NULL,
  block_label_col = NULL,
  block_lwd = 2,
  main = NULL,
  legend = TRUE,
  ...
)
```

Arguments

design	An object returned by augmented_row_column_design() or a design data frame with row, column, and block columns.
label	Cell label to show. Use "all_entries", "type", "block", or "none".
fill	Cell coloring variable. Use "type", "block", "rowgroup", "colgroup", or "is_control".
show_plot_labels	Logical. If TRUE, labels are printed inside cells unless label = "none".
show_block_labels	Logical. If TRUE, block numbers are printed. Defaults to FALSE so block labels do not overlap accession labels.
block_label_position	Position for block labels when show_block_labels = TRUE. Use "background" for a faint watermark behind accession labels or "center" for a regular center label.
label_cex	Optional text size for plot-cell labels.
block_label_cex	Optional text size for block labels.
block_label_col	Color for block labels.
block_lwd	Line width for block boundaries.
main	Optional plot title.
legend	Logical. If TRUE, draw a legend for cell colors.
...	Additional arguments passed to graphics::plot.window().

Value

Invisibly returns the plotted design data frame with internal plotting coordinates.

Examples

```
design <- augmented_row_column_design(
  treatments = paste0("G", seq_len(108)),
  controls = paste0("Check", seq_len(4)),
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 20,
  seed = 123
)

plot_augmented_row_column_design(design, label = "all_entries")
```

`plot_resolvable_row_column_design` *Plot a resolvable row-column design*

Description

Draws the field with replicate boundaries.

Usage

```
plot_resolvable_row_column_design(
  design,
  label = c("all_entries", "replicate", "none"),
  fill = c("replicate", "is_filler"),
  main = NULL,
  ...
)
```

Arguments

`design`	An object from `resolvable_row_column_design()` or its `design` data frame.
`label`	Cell label: `"all_entries"`, `"replicate"`, or `"none"`.
`fill`	Cell colouring: `"replicate"` or `"is_filler"`.
`main`	Optional title.
`...`	Passed to `plot_augmented_row_column_design()`.

Value

Invisibly, the plotted data.

Examples

```
design <- resolvable_row_column_design(paste0("T", 1:20), 2, 4, 5,
                                       n_starts = 1, n_iter = 300, seed = 1)
plot_resolvable_row_column_design(design)
```

`prediction_accuracy` *Accuracy of adjusted means against known values*

Description

In a simulation the true genotype values are known. This function measures how well an analysis recovered them: the correlation between the adjusted means and the true values.

Usage

```
prediction_accuracy(
  fit,
  truth,
  genotypes = NULL,
  method = c("pearson", "spearman")
)
```

Arguments

`fit`	An analysis object (from any `analyze_*()` function) or a data frame with `genotype` and `adjusted_mean` columns.
`truth`	Named numeric vector of true genotype values; names are the genotype names.
`genotypes`	Optional subset of genotypes to use (for example only the test entries, without the checks).
`method`	Correlation method: `"pearson"` (default) or `"spearman"` (rank correlation, i.e. how well the ranking is recovered).

Value

A single number between -1 and 1.

Examples

```
design <- augmented_prep_design(paste0("G", 1:30), c("A", "B"), block_size = 6, seed = 1)
trial <- design$design
set.seed(1)
truth <- setNames(rnorm(30, sd = 3), paste0("G", 1:30))
trial$yield <- 50 + truth[trial$all_entries] + rnorm(nrow(trial))
fit <- analyze_augmented_prep_design(trial, "yield")
prediction_accuracy(fit, truth)
```

prepare_breedbase_pheno *Prepare Breedbase phenotype data for multi-environment analysis*

Description

This function standardizes Breedbase phenotype data into five columns: `environment`, `genotype`, `replicate`, `block`, and `value`.

Usage

```
prepare_breedbase_pheno(
  pheno_raw,
  trait_col,
  location_col = NULL,
  genotype_col = NULL,
  rep_col = NULL,
  block_col = NULL
)
```

Arguments

`pheno_raw`	A raw Breedbase phenotype data frame, usually read from a Breedbase Excel file.
`trait_col`	Name of the trait column to analyze.
`location_col`	Optional location/environment column name. If `NULL`, the function tries to guess it.
`genotype_col`	Optional genotype/accession column name. If `NULL`, the function tries to guess it.
`rep_col`	Optional replicate column name.
`block_col`	Optional block column name.

Value

A tibble with standardized phenotype data.

Examples

```
df <- data.frame(
  location = c("Loc1", "Loc1", "Loc2", "Loc2"),
  germplasmName = c("G1", "G2", "G1", "G2"),
  rep = c(1, 1, 1, 1),
  phenotype = c(10, 12, 15, 14)
)

prepare_breedbase_pheno(df, trait_col = "phenotype")
```

prepare_design_factors *Prepare design factors*

Description

Prepare design factors

Usage

```
prepare_design_factors(data)
```

Arguments

data	Standardized phenotype data.

Value

Data frame with design columns converted to factors.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

str(prepare_design_factors(prepared$data))
```

prepare_environment_stratification_data *Prepare phenotype data for Lin environment stratification*

Description

Reads a Breedbase-style phenotype data frame and standardizes it for environment stratification.

Usage

```
prepare_environment_stratification_data(pheno, trait, normalize_names = TRUE)
```

Arguments

pheno	A phenotype data frame.
trait	Trait column name.
normalize_names	Logical. If TRUE, normalizes column names.

Value

A list with standardized phenotype data, environment metadata, and selected trait column.

Examples

```
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
prepared <- prepare_environment_stratification_data(pheno, trait = "fresh root yield")

head(prepared$data)
prepared$env_info
prepared$trait_col
```

---

print.augmented_prep_analysis

Print augmented p-rep analysis

Description

Print augmented p-rep analysis

Usage

```
## S3 method for class 'augmented_prep_analysis'
print(x, ...)
```

Arguments

x	An object returned by analyze_augmented_prep_design().
...	Additional arguments passed to print().

Value

Invisibly returns x.

print.augmented_prep_two_stage

Print a two-stage p-rep analysis

Description

Print a two-stage p-rep analysis

Usage

```
## S3 method for class 'augmented_prep_two_stage'
print(x, n = 10, ...)
```

Arguments

x	An object returned by analyze_augmented_prep_two_stage().
n	Number of top-ranked genotypes to show.
...	Unused.

Value

Invisibly returns x.

print.augmented_row_column_analysis

Print augmented row-column analysis

Description

Print augmented row-column analysis

Usage

```
## S3 method for class 'augmented_row_column_analysis'
print(x, ...)
```

Arguments

x	An object returned by analyze_augmented_row_column_design().
...	Additional arguments passed to print().

Value

Invisibly returns x.

print.resolvable_row_column_analysis *Print a resolvable row-column analysis*

Description

Print a resolvable row-column analysis

Usage

```
## S3 method for class 'resolvable_row_column_analysis'
print(x, n = 10, ...)
```

Arguments

x	An object returned by analyze_resolvable_row_column().
n	Number of top-ranked treatments to show.
...	Unused.

Value

Invisibly returns x.

read_breedbase_file *Read phenotype or design data from Excel, CSV, or tab-delimited files*

Description

Loads phenotype or field-design data from .xlsx, .xls, .csv, .tsv, .txt, or .design files. Excel files are read from all non-empty sheets and combined into one tibble with a .sheet column identifying the source sheet. CSV files are comma-delimited; .tsv, .txt, and .design files are tab-delimited.

Usage

```
read_breedbase_file(file, sheet = NULL, ...)
```

Arguments

file	Path to the phenotype file.
sheet	Optional Excel sheet name or number. If NULL, all non-empty Excel sheets are combined.
...	Additional arguments passed to readxl::read_excel() for Excel files, utils::read.csv() for CSV files, or utils::read.table() for tab-delimited files.

Value

A tibble containing the loaded phenotype data.

Examples

```
# CSV phenotype download
pheno_csv <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
head(pheno_csv)

# Excel workbook: all non-empty sheets are combined
pheno_xlsx <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.xlsx", package = "breedtools")
)
table(pheno_xlsx$.sheet)

# A single sheet
read_breedbase_file(
  system.file("extdata", "breedbase_pheno.xlsx", package = "breedtools"),
  sheet = 1
)
```

---

read_locations *Load location coordinates*

Description

Loads location metadata from a `.xlsx`, `.xls`, or `.csv` file, or from an already loaded data frame. The returned table has standardized columns: `location`, `latitude`, and `longitude`.

Usage

```
read_locations(
  file,
  location_col = NULL,
  latitude_col = NULL,
  longitude_col = NULL,
  phenotype_data = NULL,
  phenotype_location_col = NULL,
  match_by = "location",
  reference_locations = NULL,
  ignore_case = FALSE,
  sheet = NULL,
  ...
)
```

Arguments

file	Path to a location file, or an already loaded data frame.
location_col	Optional location name column.
latitude_col	Optional latitude column.
longitude_col	Optional longitude column.
phenotype_data	Optional raw phenotype data frame, prepared phenotype data frame, or result from run_environment_stratification(). If supplied, location names are extracted and checked against the location file.
phenotype_location_col	Optional location column in phenotype_data.
match_by	Column to extract when phenotype_data is a stratification result. Use "location", "environment", or "environment_label".
reference_locations	Optional character vector of expected location names. If supplied, every reference location must be present in file.
ignore_case	Logical. If TRUE, location matching ignores case.
sheet	Optional Excel sheet name or number passed to read_breedbase_file().
...	Additional arguments passed to read_breedbase_file().

Value

A tibble with location, latitude, and longitude.

Examples

```
locations <- data.frame(
  location = c("Loc1", "Loc2"),
  latitude = c(35.1, 36.2),
  longitude = c(-78.9, -79.4)
)

read_locations(locations)

# Breedbase-style location file, checked against the phenotype locations
loc_file <- system.file("extdata", "breedbase_locations.csv", package = "breedtools")
pheno <- read_breedbase_file(
  system.file("extdata", "breedbase_pheno.csv", package = "breedtools")
)
read_locations(loc_file, phenotype_data = pheno)
```

`resolvable_row_column_design` *Generate a resolvable (spatial) row-column design*

Description

Builds a resolvable row-column design for `v <= k * s` treatments in `r` replicates, each a `k x s` array of rows and columns that contains every treatment once (Williams, John and Whitaker, 2006). The design is found by a search over swaps of treatments within replicates:

1. The average efficiency factor of the resolvable row-column design (rows and columns within replicates), together with the efficiency factors of the component row design and column design, is maximized.
2. With `spatial = TRUE`, the spatial average efficiency factor of the two-dimensional linear variance (LV + RC) model is then maximized while keeping the three efficiency factors of step 1 within `efficiency_tolerance` of their optimized values. This spreads treatments over edge and interior plots and over row and column neighbours, without needing prior values for spatial parameters.

When replicates are contiguous, `latinize = TRUE` builds a latinized design: no treatment appears twice in the same long column (replicates stacked vertically) or long row (replicates side by side), within every group of `latin_t` adjacent replicates (t-latinized designs; John and Williams, 1998).

Usage

```
resolvable_row_column_design(
  treatments,
  n_reps,
  rows_per_rep,
  cols_per_rep,
  spatial = TRUE,
  latinize = FALSE,
  replicate_layout = c("vertical", "horizontal"),
  latin_t = NULL,
  n_starts = 3,
  n_iter = NULL,
  efficiency_tolerance = 0.002,
  plot_type = c("serpentine", "cartesian"),
  seed = NULL,
  output_file = NULL,
  start_design = NULL
)
```

Arguments

`treatments`	Character vector of treatment names (`v <= k * s`).
`n_reps`	Number of replicates (`r`).

rows_per_rep	Number of rows in each replicate (k).
cols_per_rep	Number of columns in each replicate (s).
spatial	Logical. If TRUE (default), optimize the spatial efficiency factor in a second stage.
latinize	Logical. If TRUE, build a latinized design for contiguous replicates.
replicate_layout	How replicates are placed in the field: "vertical" (stacked, giving long columns) or "horizontal" (side by side, giving long rows).
latin_t	Optional size of the groups of adjacent replicates within which latinization holds (t-latinized design). Defaults to all replicates.
n_starts	Number of random starting designs.
n_iter	Number of swap proposals per stage and start. Defaults to min(20000, max(3000, 60 * v * r)).
efficiency_tolerance	Largest loss in the row-column, row and column efficiency factors accepted in the spatial stage.
plot_type	Plot numbering order: "serpentine" or "cartesian".
seed	Optional random seed.
output_file	Optional tab-delimited output file for the design.
start_design	Optional existing design to start from (for example one produced by CycDesigN), as a data frame with replicate, row_in_rep, col_in_rep and all_entries columns and complete replicates. The search then improves this design; use n_iter = 1 and spatial = FALSE to keep it essentially as it is and only lay it out.

Details

When there are fewer treatments than plots per replicate, the remaining plots are fillers (all_entries = "FILLER", is_filler = 1), placed by the search like an extra treatment.

Value

An object of class resolvable_row_column_design: a list with input, summary (efficiency factors, spatial upper bound, latinization check), design (one row per plot with replicate, row_in_rep, col_in_rep, field row and col, long_row / long_col, all_entries, is_filler), breedbase_design, and efficiency (see resolvable_row_column_efficiency()).

References

Williams, E. R., John, J. A. and Whitaker, D. (2006). Construction of resolvable spatial row-column designs. Biometrics 62, 103-108. doi:10.1111/j.15410420.2005.00393.x

John, J. A. and Williams, E. R. (1998). t-Latinized designs. Australian and New Zealand Journal of Statistics 40, 111-118.

See Also

resolvable_row_column_efficiency(), analyze_resolvable_row_column(), plot_resolvable_row_column_desig

Examples

```
# Example 2 of Williams, John and Whitaker (2006): 42 treatments in three
# replicates of 6 rows x 7 columns.
design <- resolvable_row_column_design(
  treatments = paste0("T", 1:42),
  n_reps = 3,
  rows_per_rep = 6,
  cols_per_rep = 7,
  n_starts = 1,
  n_iter = 2000,
  seed = 1
)
design$summary
```

---

resolvable_row_column_efficiency

Efficiency of a resolvable row-column design

Description

Computes the design criteria of Williams, John and Whitaker (2006) for a resolvable row-column design with complete replicates: the average efficiency factors of the row-column design, the row design and the column design (rows and columns nested within replicates), the spatial average efficiency factor of the two-dimensional linear variance model,

$$A = rI - X'(I_r \otimes W_R \otimes W_C)X,$$

with its upper bound $U = (v - t)/(v - 1)$, $t = tr(W_R)tr(W_C)$, and treatment neighbour and edge statistics. Optionally, it evaluates the average pairwise variance criterion $z(X)$ under an assumed autoregressive (AR) model, which is used to compare designs (Section 4 of the paper).

Usage

```
resolvable_row_column_efficiency(
  design,
  ar = NULL,
  genotype_col = "all_entries",
  replicate_col = "replicate",
  row_col = "row_in_rep",
  col_col = "col_in_rep"
)
```

Arguments

design	A design object from resolvable_row_column_design() or a data frame with replicate, row, column and treatment columns. Each replicate must be complete (k * s plots, each treatment once; filler plots count as separate treatments).

ar — Optional named list of AR model parameters eta_r, eta_c, psi, rho_r, rho_c (Eq. 4 of the paper).

genotype_col, replicate_col, row_col, col_col — Column names.

Value

A list with efficiency_rc, efficiency_rows, efficiency_cols, efficiency_spatial, upper_bound_spatial, ar_criterion (NA unless ar is given), and neighbours, a tibble with each treatment's number of distinct row and column neighbours and the number of replicates in which it is on an edge or in a corner.

Examples

```
design <- resolvable_row_column_design(paste0("T", 1:20), 2, 4, 5,
                                       n_starts = 1, n_iter = 500, seed = 1)
eff <- resolvable_row_column_efficiency(design,
  ar = list(eta_r = 35, eta_c = 60, psi = 10, rho_r = 0.5, rho_c = 0.5))
eff[c("efficiency_rc", "efficiency_spatial", "ar_criterion")]
```

---

run_breedbase_environment_stratification

Run Lin-style environment stratification from a Breedbase phenotype file

---

Description

Reads a Breedbase phenotype file, prepares the phenotype data, detects the experimental design, calculates ANOVA, and groups environments using a Lin-style genotype-by-environment interaction test.

Usage

```
run_breedbase_environment_stratification(
  file,
  trait_col,
  alpha = 0.05,
  normalize_names = TRUE
)
```

Arguments

file — Path to a .xlsx, .xls, or .csv phenotype file, or an already loaded data frame.

trait_col — Trait column name.

alpha — Significance level. Default is 0.05.

normalize_names — Logical. If TRUE, normalize Breedbase-style column names.

Value

A list containing standardized phenotype data, environment metadata, ANOVA results, pairwise environment tests, group summary, group membership, ungrouped environments, summary, and message.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")

result <- run_breedbase_environment_stratification(
  file = pheno_file,
  trait_col = "fresh root yield",
  alpha = 0.05
)

result$group_summary
result$group_membership
```

---

run_breedbase_variance_check

Run Breedbase multi-location phenotype variance check

Description

Reads a Breedbase phenotype file, runs individual ANOVA by environment, performs Hartley Fmax tests, and identifies compatible environment groups for joint analysis.

Usage

```
run_breedbase_variance_check(
  file,
  trait_col,
  alpha = 0.05,
  location_col = NULL,
  genotype_col = NULL,
  rep_col = NULL,
  block_col = NULL,
  max_group_size = Inf,
  n_sim = 1e+05,
  seed = 123,
  export = FALSE,
  output_dir = NULL
)
```

Arguments

file	Path to a .xlsx, .xls, or .csv phenotype file, or an already loaded data frame.
trait_col	Trait column to analyze.
alpha	Significance level for Hartley Fmax test.
location_col	Optional location/environment column.
genotype_col	Optional genotype/accession column.
rep_col	Optional replicate column.
block_col	Optional block column.
max_group_size	Maximum environment group size to test.
n_sim	Number of simulations for Hartley critical value.
seed	Random seed for the Hartley critical value simulation. The user's global random number generator state is not modified.
export	Logical. If TRUE, writes CSV files to output_dir.
output_dir	Directory where CSV files should be written. Defaults to NULL, in which case nothing is written; it must be supplied when export = TRUE.

Value

A list containing cleaned data, ANOVA results, Hartley test, all group tests, compatible groups, maximal compatible groups, and detected columns.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")

result <- run_breedbase_variance_check(
  file = pheno_file,
  trait_col = "fresh root yield|CO_334:0000013",
  n_sim = 2000
)

result$env_anova
result$maximal_compatible_groups

# Export CSV files to a temporary directory
out_dir <- file.path(tempdir(), "variance_check")
result <- run_breedbase_variance_check(
  file = pheno_file,
  trait_col = "fresh root yield|CO_334:0000013",
  n_sim = 2000,
  export = TRUE,
  output_dir = out_dir
)
list.files(out_dir)
unlink(out_dir, recursive = TRUE)
```

run_environment_stratification *Run Lin-style environment stratification*

Description

This function reads phenotype data, detects design structure, calculates ANOVA, tests genotype-by-environment interaction for environment groups, and returns compatible environment groups.

Usage

```
run_environment_stratification(
  phenotype_file,
  trait,
  alpha = 0.05,
  sep = NULL,
  normalize_names = TRUE,
  export_json = FALSE,
  output_dir = NULL,
  prefix = "environment_stratification"
)
```

Arguments

phenotype_file	Path to a .xlsx, .xls, or .csv phenotype file, or an already loaded data frame.
trait	Trait column name.
alpha	Significance level. Default is 0.05.
sep	Deprecated. File delimiters are detected from the extension by read_breedbase_file().
normalize_names	Logical. If TRUE, normalize Breedbase column names.
export_json	Logical. If TRUE, writes a JSON result file to output_dir.
output_dir	Output directory for the JSON file. Defaults to NULL, in which case nothing is written; it must be supplied when export_json = TRUE.
prefix	Prefix for JSON output files.

Value

A list with summary, ANOVA results, pairwise tests, group summary, group membership, ungrouped environments, and message.

Examples

```
pheno_file <- system.file("extdata", "breedbase_pheno.csv", package = "breedtools")

result <- run_environment_stratification(
```

```
  phenotype_file = pheno_file,
  trait = "fresh root yield",
  alpha = 0.05
)

result$group_summary
result$group_membership

# Export the result as JSON to a temporary directory
out_dir <- file.path(tempdir(), "stratification")
result <- run_environment_stratification(
  phenotype_file = pheno_file,
  trait = "fresh root yield",
  export_json = TRUE,
  output_dir = out_dir
)
list.files(out_dir)
unlink(out_dir, recursive = TRUE)
```

---

save_breedbase *Save a design to Breedbase*

Description

Creates the trial and its plots in a Breedbase database through BrAPI, in a breeding program and, optionally, a folder of your choice. Works for designs from augmented_row_column_design(), augmented_prep_design() and resolvable_row_column_design().

Usage

```
save_breedbase(
  design,
  con = NULL,
  program = NULL,
  folder = NULL,
  trial_name,
  year,
  location = NULL,
  description,
  trial_type = "phenotyping_trial",
  design_type = NULL,
  plot_names = NULL,
  plot_name_pattern = "{trial}_{accession}_{block}_{rep}",
  create_folder = FALSE,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

design	A design object.
con	A connection from breedbase_login().
program	Breeding program name.
folder	Optional folder inside the program. Without a folder (the default, NULL) the trial is saved directly in the breeding program. An existing folder is used as it is; a new one is created when create_folder = TRUE.
trial_name	Name of the new trial (must not exist yet).
year	Year of the trial.
location	Breedbase location name. For p-rep designs, a named vector mapping the design's locations to Breedbase locations, e.g. c(North = "Ibadan", South = "Ubiaja"); by default the design's location names are used.
description	Description of the trial (required), e.g. its purpose, material and season: "Advanced yield trial of 108 lines and 4 checks, Ibadan 2026". It is shown on the trial page in Breedbase.
trial_type	Breedbase trial type, e.g. "Preliminary Yield Trial". Types the database does not know are saved as a miscellaneous trial.
design_type	Breedbase design type. Defaults to "Augmented" for augmented row-column designs, "p-rep" for p-rep designs and "Alpha" (rows as incomplete blocks within replicates) for resolvable row-column designs.
plot_names	Optional plot names given by you: either a character vector with one name per plot, in the row order of design$design, or the name of a column of design$design that holds them. Names must be unique.
plot_name_pattern	Pattern for automatic plot names, used when plot_names is NULL. The default "{trial}_{accession}_{block}_{rep}" gives names like "AYT_2026_LINE_007_3_1". Available fields: {trial} (the Breedbase trial name), {accession}, {block}, {rep}, {plot} (plot number), {row} and {col}. If a pattern gives the same name to two plots (the same accession twice in one block and rep), the plot number is added to those names.
create_folder	Logical. Create folder if it does not exist.
dry_run	Logical. Make all checks but write nothing.
confirm	Logical. Ask before writing (default in interactive sessions).

Details

Before anything is written, every name is checked: the breeding program, the folder if given (created when create_folder = TRUE), the location(s), that every entry and check already exists as an accession, and that the trial name is free. In an interactive session you are then asked to confirm. After saving, the plots are read back from Breedbase to verify the upload.

Each plot is saved with its name, plot number, row and column, rep, block and whether it is a check. Plot names are made automatically from the trial name, accession, block and rep (see plot_name_pattern), or you can give your own with plot_names. Plot names must be unique in the whole database, which is checked before saving. A multi-location p-rep design becomes

one Breedbase trial per location, named "<trial_name>_<location>". Filler plots of resolvable row-column designs are not saved.

Run first with dry_run = TRUE: all the checks are made, but nothing is written. Without a connection, dry_run = TRUE returns the data that would be sent, without contacting any server.

Value

Invisibly, a tibble with one row per Breedbase trial: location, trial name, trial id, number of plots sent and found after saving, and the address of the trial page. With dry_run = TRUE and no connection, a list with the trial and plot data that would be sent.

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so save_breedbase() after breedbase_check() does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run save_breedbase() again: an existing folder is used, and a trial left without plots gets its plots. Settings: options(breedtools.write_wait = 30, breedtools.write_checks = 10). Settings: options(breedtools.germplasm_batch = 10) for smaller batches, options(breedtools.breedbase_timeout = 600) to wait longer (seconds) for each answer.

See Also

breedbase_login(), breedbase_programs()

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
# What would be sent (no server needed):
payload <- save_breedbase(rc, trial_name = "AYT_2026_Ithaca", year = 2026,
                          location = "Ithaca",
                         description = "Advanced yield trial of 108 lines and 4 checks",
                          dry_run = TRUE)
str(payload$studies[[1]], max.level = 1)
payload$plots[[1]][[1]]

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase(rc, con, program = "IITA", folder = "AYT 2026",
               trial_name = "AYT_2026_Ibadan", year = 2026, location = "Ibadan",
              description = "Advanced yield trial of 108 lines and 4 checks, Ibadan 2026",
               create_folder = TRUE, dry_run = TRUE)   # check first
save_breedbase(rc, con, program = "IITA", folder = "AYT 2026",
```

```
                trial_name = "AYT_2026_Ibadan", year = 2026, location = "Ibadan",
              description = "Advanced yield trial of 108 lines and 4 checks, Ibadan 2026")

## End(Not run)
```

`save_breedbase_analysis` *Save second-stage results to Breedbase as an analysis*

Description

Saves genotype-level results (for example adjusted means across locations from `second_stage_values()`) as a Breedbase analysis, the way the Breedbase mixed-models tool does: a new project of type analysis with one entry per accession, and each trait saved together with a statistic term. The analysis then appears on the Analyses page of Breedbase.

Usage

```
save_breedbase_analysis(
  data,
  con = NULL,
  analysis_name,
  program,
  description,
  year = format(Sys.Date(), "%Y"),
  traits = NULL,
  accession = "accession",
  estimate = NULL,
  statistic = NULL,
  model_type = "mixed_model_lmer",
  protocol = NULL,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

`data`	A data frame with one row per accession and one numeric column per trait (as from `second_stage_values()`).
`con`	A connection from `breedbase_login()`; `NULL` with `dry_run = TRUE` returns what would be sent.
`analysis_name`	Name of the new analysis (must not exist yet).
`program`	Breeding program of the analysis.
`description`	Description of the analysis (required).
`year`	Year of the analysis (default: this year).

`traits` Trait columns (default: all except `accession`). Their names are the Breedbase trait names.

`accession` Name of the accession column.

`estimate` `"blue"` (statistic "Adjusted means from BLUEs using LMER R") or `"blup"` (statistic "Adjusted means from BLUPs using LMER R"). By default taken from `second_stage_values()`, else `"blue"`.

`statistic`, `model_type` To use another Breedbase statistic term (`"name|SGNSTAT:0000000"`) or model type (a `protocol_type` term; default `"mixed_model_lmer"`).

`protocol` Text describing the model, saved with the analysis (default: the description).

`dry_run` Logical. Check everything but write nothing.

`confirm` Logical. Ask before writing (default in interactive sessions).

Details

Every accession and trait is checked first, and all problems are listed before anything is written.

Value

Invisibly, a tibble with the analysis name and id, the numbers of accessions and values sent, the accessions read back, and the analysis page.

Examples

```
prep <- augmented_prep_design(paste0("E", 1:60), locations = c("North", "South"), seed = 1)
d <- prep$design
set.seed(1)
d$yield <- rnorm(nrow(d), 25, 3)
fit <- analyze_augmented_prep_two_stage(d, "yield")
means <- second_stage_values(fit, trait = "fresh root yield")
sent <- save_breedbase_analysis(means, analysis_name = "PREP_2026_stage2", program = "IITA",
                            description = "Two-stage analysis, weighted", dry_run = TRUE)
str(sent$analysis_result_values[1])

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase_analysis(means, con, analysis_name = "PREP_2026_stage2", program = "IITA",
                        description = "Two-stage analysis, weighted")

## End(Not run)
```

save_breedbase_first_stage *Save first-stage results to Breedbase as a trait of the field plots*

Description

Saves plot-level first-stage results (from `first_stage_values()`) on the plots of the field trial, as values of a trait made for adjusted values, so raw and adjusted values stay apart. By default that trait is the measured trait composed with a statistic, e.g. "fresh root yield|Adjusted Means from Spatial Correction using SpATS R": it is found, or created (after you confirm) when it does not exist yet; see `breedbase_adjusted_trait()`. Or name your own traits with `as_trait`. The values are saved through BrAPI, like `save_breedbase_phenotypes()`.

Usage

```
save_breedbase_first_stage(
  values,
  con = NULL,
  as_trait = NULL,
  statistic = "Adjusted Means from Spatial Correction using SpATS R",
  create_trait = TRUE,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

`values`	A table from `first_stage_values()` (or any table with a `plot_name` column and one numeric column per trait).
`con`	A connection from `breedbase_login()`; `NULL` with `dry_run = TRUE` returns what would be sent.
`as_trait`	Optional named vector: for each trait column of `values`, an existing Breedbase trait to save it under, e.g. `c("fresh root yield" = "fresh root yield adjusted|COMP:0000123")`.
`statistic`	The statistic composed with each trait when `as_trait` is not given (see `breedbase_adjusted_trait()`).
`create_trait`	Logical. Create the composed traits that do not exist yet (you are asked first when `confirm = TRUE`).
`dry_run, confirm`	As in `save_breedbase_phenotypes()`.

Value

As `save_breedbase_phenotypes()`: one row per trait with the values sent and found.

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so `save_breedbase()` after `breedbase_check()` does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run `save_breedbase()` again: an existing folder is used, and a trial left without plots gets its plots. Settings: `options(breedtools.write_wait = 30, breedtools.write_checks = 10)`. Settings: `options(breedtools.germplasm_batch = 10)` for smaller batches, `options(breedtools.breedbase_timeout = 600)` to wait longer (seconds) for each answer.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
d <- rc$design
set.seed(1)
d$yield <- 25 + 0.4 * d$row + rnorm(nrow(d), sd = 2)
fit <- analyze_augmented_row_column_design(d, "yield")
values <- first_stage_values(fit, rc, "AYT_2026_Ibadan", trait = "fresh root yield")
obs <- save_breedbase_first_stage(values, dry_run = TRUE)
head(obs)

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase_first_stage(values, con, dry_run = TRUE)  # check; shows the trait to create
save_breedbase_first_stage(values, con)              # creates the trait if needed, saves

## End(Not run)
```

---

save_breedbase_phenotypes

Save phenotypic data to Breedbase

Description

Uploads trait values of plots that are already in Breedbase (for example saved with `save_breedbase()`) through BrAPI. Give the plot name and the trait name with each value; the plots and traits are looked up in Breedbase first, and every problem is listed before anything is written.

Usage

```
save_breedbase_phenotypes(
  data,
  con = NULL,
  plot = "plot_name",
  traits = NULL,
  trait = NULL,
  value = NULL,
  timestamp = NULL,
  dry_run = FALSE,
  confirm = interactive()
)
```

Arguments

data	A data frame with the plot names and the values.
con	A connection from breedbase_login(); NULL with dry_run = TRUE returns the observations without checking them.
plot	Name of the column with the plot names.
traits	For wide data: the trait columns. By default every column except plot, timestamp and the design columns of phenotype_template() (trial_name, accession, plot_number, block, rep, row, col, is_check).
trait, value	For long data: the names of the trait and value columns.
timestamp	Optional name of a column with the date and time of each measurement (e.g. "2026-08-15 10:30:00"). Breedbase uses it to tell repeated measurements apart.
dry_run	Logical. Check everything but write nothing.
confirm	Logical. Ask before writing (default in interactive sessions).

Details

The data can be wide, one row per plot and one column per trait (as made by phenotype_template()):

plot_name	fresh root yield	dry matter content percentage
AYT_2026_Ibadan_G31_1_1	25.4	31.2

or long, one row per value, with trait and value columns (trait = "trait", value = "value").

Trait names are the Breedbase names, with or without the ontology id: "fresh root yield" or "fresh root yield|CO_334:0000013" (the column names of a Breedbase phenotype download). Empty values (NA or "") are skipped. Values are checked by Breedbase against the trait (e.g. numbers within the allowed range).

Value

Invisibly, a tibble with one row per trait: trait, observation_variable_db_id, values_sent and values_found (read back from Breedbase after saving).

Slow servers

Accessions are searched 25 names at a time, because Breedbase looks up the pedigree of each one. If the server times out (HTTP 504, 502, 503 or 429), the batch is split in half and tried again, so a busy server is slower but still works. A line is printed after each batch with the names done and the time. Accessions found are remembered until R is restarted, so `save_breedbase()` after `breedbase_check()` does not search them again.

When creating the folder, the trial or its plots times out, the save does not stop: the server may still finish the job, so it checks for the result every 20 seconds for 3 minutes, and tries once more if it is not there (Breedbase refuses names that already exist, so this cannot make duplicates). If a save is cut off anyway, run `save_breedbase()` again: an existing folder is used, and a trial left without plots gets its plots. Settings: `options(breedtools.write_wait = 30, breedtools.write_checks = 10)`. Settings: `options(breedtools.germplasm_batch = 10)` for smaller batches, `options(breedtools.breedbase_timeout = 600)` to wait longer (seconds) for each answer.

Examples

```
rc <- augmented_row_column_design(paste0("G", 1:108), paste0("C", 1:4),
                                  12, 12, 4, 4, n_candidates = 5, seed = 1)
sheet <- phenotype_template(rc, "AYT_2026_Ibadan", traits = "fresh root yield")
sheet$`fresh root yield` <- round(rnorm(nrow(sheet), 25, 4), 1)

# What would be sent (no connection needed):
obs <- save_breedbase_phenotypes(sheet, dry_run = TRUE)
head(obs)

## Not run:
con <- breedbase_login("https://cassavabase.org", "jdoe")
save_breedbase_phenotypes(sheet, con, dry_run = TRUE)   # check first
save_breedbase_phenotypes(sheet, con)

# Long data: one row per value
long <- data.frame(plot_name = sheet$plot_name, trait = "fresh root yield",
                   value = sheet$`fresh root yield`)
save_breedbase_phenotypes(long, con, trait = "trait", value = "value")

## End(Not run)
```

---

second_stage_values *Second-stage genotype values from analyses*

Description

Collects the adjusted means of analyses (one per trait) into one table with one row per accession, ready for `save_breedbase_analysis()`.

Usage

```
second_stage_values(fits, trait = NULL)
```

Arguments

fits	One analysis, or a named list of analyses, one per trait (the names are the Breedbase trait names). Any breedtools analysis with adjusted_means, e.g. analyze_augmented_prep_two_stage().
trait	Trait name when fits is a single analysis.

Value

A tibble with accession and one column per trait. Its attribute "estimate" is "blup" when the genotypes were random, else "blue".

Examples

```
prep <- augmented_prep_design(paste0("E", 1:60), locations = c("North", "South"), seed = 1)
d <- prep$design
set.seed(1)
d$yield <- rnorm(nrow(d), 25, 3)
fit <- analyze_augmented_prep_two_stage(d, "yield")
head(second_stage_values(fit, trait = "fresh root yield"))
```

Index

add_environment_metadata, 6
add_pairwise_environment_metadata, 6
analyze_augmented_prep_design, 7
analyze_augmented_prep_design(), *4*, *10–12*, *22*, *41*
analyze_augmented_prep_two_stage, 10
analyze_augmented_prep_two_stage(), *4*, *9*, *41*, *80*
analyze_augmented_row_column_design, 13
analyze_augmented_row_column_design(), *4*, *41*
analyze_environment_anova, 15
analyze_environment_anova(), *48*
analyze_one_environment, 16
analyze_resolvable_row_column, 17
analyze_resolvable_row_column(), *5*, *41*, *61*, *65*
augmented_prep_concurrence, 19
augmented_prep_concurrence(), *4*
augmented_prep_design, 19
augmented_prep_design(), *4*, *10, 11*, *31*, *34*, *51*, *71*
augmented_prep_parameters, 23
augmented_prep_parameters(), *4*, *22*
augmented_row_column_capacity, 24
augmented_row_column_capacity(), *4*, *26*, *28, 29*
augmented_row_column_design, 25
augmented_row_column_design(), *4*, *29*, *31*, *34*, *51*, *71*
augmented_row_column_layouts, 28
augmented_row_column_layouts(), *4*

breedbase_adjusted_trait, 29
breedbase_adjusted_trait(), *5*, *76*
breedbase_check, 30
breedbase_check(), *5*, *31*, *73*, *77*, *79*
breedbase_folders *(*breedbase_lists*)*, 32
breedbase_folders(), *5*, *33*
breedbase_lists, 32
breedbase_locations *(*breedbase_lists*)*, 32
breedbase_locations(), *5*, *33*
breedbase_login, 33
breedbase_login(), *5*, *30–32*, *72–74*, *76*, *78*
breedbase_programs *(*breedbase_lists*)*, 32
breedbase_programs(), *5*, *33*, *73*
breedtools *(*breedtools-package*)*, 4
breedtools-package, 4

calculate_environment_stratification_anova, 34
check_design, 34
check_design(), *5*, *35–38*
check_reset, 35
check_reset(), *36*
check_result, 36
check_result(), *5*, *35*, *37, 38*
check_section, 37
check_section(), *34*, *36*
check_setup, 37
check_setup(), *5*
check_summary, 38
check_summary(), *5*, *34–38*
complete_environment_info, 39

detect_design, 39

environment_summary_by_group, 40

first_stage_values, 41
first_stage_values(), *5*, *76*

get_compatible_groups, 42
get_compatible_groups(), *43*
get_maximal_groups, 42
guess_column, 43

hartley_critical_sim, 44
hartley_critical_sim(), *45*

hartley_fmax_test, 44
hartley_fmax_test(), *5*

leaflet::addProviderTiles(), *50*
lin_error_mse, 45
lin_group_environments, 46
lin_group_environments(), *5*
lin_ss_ge, 47
lin_test_group, 47

make_environment_groups, 48
make_environment_groups(), *42*
map_environment_groups, 49
map_environment_groups(), *5*

phenotype_template, 50
phenotype_template(), *5*, *78*
plot_augmented_prep_design, 51
plot_augmented_prep_design(), *4*, *22*
plot_augmented_row_column_design, 53
plot_augmented_row_column_design(), *4*, *55*
plot_resolvable_row_column_design, 55
plot_resolvable_row_column_design(), *5*, *65*
prediction_accuracy, 56
prediction_accuracy(), *5*
prepare_breedbase_pheno, 57
prepare_breedbase_pheno(), *5*, *15*
prepare_design_factors, 58
prepare_environment_stratification_data, 58
print.augmented_prep_analysis, 59
print.augmented_prep_two_stage, 60
print.augmented_row_column_analysis, 60
print.resolvable_row_column_analysis, 61

read_breedbase_file, 61
read_breedbase_file(), *5*
read_locations, 62
read_locations(), *5*
resolvable_row_column_design, 64
resolvable_row_column_design(), *5*, *17*, *31*, *34*, *51*, *55*, *66*, *71*
resolvable_row_column_efficiency, 66
resolvable_row_column_efficiency(), *5*, *65*
run_breedbase_environment_stratification, 67
run_breedbase_variance_check, 68
run_breedbase_variance_check(), *5*
run_environment_stratification, 70
run_environment_stratification(), *5*

save_breedbase, 71
save_breedbase(), *5*, *30–33*, *41*, *50, 51*, *73*, *77*, *79*
save_breedbase_analysis, 74
save_breedbase_analysis(), *5*, *79*
save_breedbase_first_stage, 76
save_breedbase_first_stage(), *5*, *41*
save_breedbase_phenotypes, 77
save_breedbase_phenotypes(), *5*, *50*, *76*
second_stage_values, 79
second_stage_values(), *5*, *74, 75*